

Planeación de acciones para robots de servicio
mediante modelos de lenguaje y sistemas basados en
reglas

Planeación de acciones para robots de servicio a
partir de instrucciones en lenguaje natural mediante
Qwen, CLIPS y modelos GPT

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción	1
1.1. Contexto y Motivación	2
1.2. Planteamiento del problema	3
1.3. Hipótesis	4
1.4. Objetivos	5
1.5. Estructura del documento	6
2. Antecedentes y Estado del Arte	8
2.1. Sistemas Basados en Reglas	8
2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones	9
2.2.1. Características Fundamentales de CLIPS	10
2.3. Modelos de lenguaje para interpretación de instrucciones robóticas	11
2.3.1. Familia de modelos Qwen	12
2.3.2. Modelos GPT	13
2.3.3. Interpretación semántica y generación de planes robóticos	14
2.3.4. Limitaciones de los modelos de lenguaje en robótica	15
2.3.5. Salidas estructuradas y validación	15
2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA	16
2.4. Arquitecturas neuro-simbólicas	16
2.5. Síntesis y conexión con la propuesta	19
3. Formalización del problema y arquitectura conceptual	21
3.1. Contexto arquitectónico y alcance del sistema	22
3.1.1. Integración dentro de la arquitectura ViRBot	22
3.1.2. Alcance del problema abordado	23
3.2. GPSR como dominio de referencia	23

3.2.1.	La prueba General Purpose Service Robot	23
3.2.2.	Generación de comandos y conocimiento del dominio	25
3.2.3.	Dominio de referencia y alcance lingüístico	26
3.3.	Formalización de órdenes, objetivos y planes	28
3.3.1.	Orden en lenguaje natural	28
3.3.2.	Equivalencia semántica entre formulaciones	29
3.3.3.	Entidades y conocimiento del dominio	31
3.3.4.	Representación canónica de objetivos	33
3.3.5.	Objetivos semánticos y acciones ejecutables	36
3.3.6.	Dependencias y contexto entre objetivos	38
3.4.	Arquitecturas conceptuales evaluadas	42
3.4.1.	Base común de las arquitecturas evaluadas	43
3.4.2.	Arquitectura Qwen-CLIPS	44
3.4.3.	Arquitectura de planificación directa con GPT	48
3.4.4.	Comparación conceptual de las arquitecturas	53
3.5.	Criterios conceptuales de validez	55
3.5.1.	Niveles de validez	56
3.5.2.	Validez de la interpretación semántica	57
3.5.3.	Validez y consistencia del plan	59
3.5.4.	Validez de la respuesta del sistema	61
3.5.5.	Condición de control para la etapa simbólica	63
3.5.6.	Planificación y ejecución física	64
4.	Metodología y diseño de las arquitecturas evaluadas	66
4.1.	Diseño metodológico de la investigación	67
4.1.1.	Problema experimental	67
4.1.2.	Variables y condiciones de comparación	67
4.1.3.	Estrategia general de evaluación	67
4.2.	Construcción del dominio experimental	67
4.2.1.	Fuentes de conocimiento GPSR	67
4.2.2.	Catálogos de personas, objetos y ubicaciones	67
4.2.3.	Familias de tareas	67
4.2.4.	Representación de los objetivos de referencia	67
4.3.	Construcción del conjunto de datos	67
4.3.1.	Generación de órdenes	67

4.3.2.	Generación de objetivos canónicos	67
4.3.3.	Variación lingüística	67
4.3.4.	Balance y deduplicación	67
4.3.5.	División de entrenamiento y evaluación	67
4.3.6.	Validación previa del conjunto de datos	67
4.4.	Diseño de la arquitectura Qwen-CLIPS	67
4.4.1.	Función asignada a Qwen	67
4.4.2.	Especialización para interpretación semántica	67
4.4.3.	Interfaz entre objetivos y planificación simbólica	67
4.4.4.	Función asignada a CLIPS	67
4.5.	Diseño de la arquitectura GPT directo	67
4.5.1.	Funciones asignadas al modelo	67
4.5.2.	Conocimiento y restricciones proporcionadas	67
4.5.3.	Contrato de salida	67
4.5.4.	Plan ejecutable, aclaración y rechazo	67
4.6.	Diseño de la comparación	67
4.6.1.	Igualdad del dominio de evaluación	67
4.6.2.	Igualdad del repertorio de acciones	67
4.6.3.	Separación entre interpretación y planificación	67
5.	Implementación	68
5.1.	Plataforma experimental	69
5.1.1.	Robot Justina	69
5.1.2.	Plataforma de cómputo	69
5.1.3.	Entorno de software y ROS 2	69
5.2.	Implementación del conjunto de datos	69
5.2.1.	Generador de órdenes	69
5.2.2.	Catálogos y CompetitionTemplate	69
5.2.3.	Validadores	69
5.2.4.	Dataset final	69
5.3.	Entrenamiento e inferencia de Qwen2.5-7B	69
5.3.1.	Preparación del modelo	69
5.3.2.	Configuración QLoRA	69
5.3.3.	Hiperparámetros de entrenamiento	69
5.3.4.	Selección del checkpoint	69

5.3.5.	Inferencia	69
5.3.6.	Despliegue del modelo	69
5.4.	Implementación del planificador CLIPS	69
5.4.1.	Conversión de objetivos a hechos gpsr-goal	69
5.4.2.	Representación del estado simbólico	69
5.4.3.	Reglas de expansión	69
5.4.4.	Generación de acciones	69
5.4.5.	Condición de finalización	69
5.5.	Integración Qwen-CLIPS con Justina	69
5.5.1.	Componentes ROS 2	69
5.5.2.	Flujo de comunicación	69
5.5.3.	Comunicación con el ejecutor	69
5.6.	Implementación del planificador GPT	69
5.6.1.	Interfaz con la API	69
5.6.2.	System prompt	69
5.6.3.	JSON Schema y Structured Outputs	69
5.6.4.	Generación de objetivos y planes	69
5.6.5.	Tratamiento de aclaraciones y rechazos	69
5.6.6.	Validación determinista	69
5.7.	Reproducibilidad	69
5.7.1.	Versiones de modelos	69
5.7.2.	Configuraciones experimentales	69
5.7.3.	Registro de ejecuciones	69
6.	Validación y experimentación	70
6.1.	Diseño experimental	71
6.1.1.	Conjunto de prueba	71
6.1.2.	Procedimiento experimental	71
6.1.3.	Repeticiones y control de variabilidad	71
6.2.	Condiciones experimentales	71
6.2.1.	Arquitectura Qwen-CLIPS	71
6.2.2.	Arquitectura GPT directo	71
6.2.3.	Condición de control con objetivos de referencia y CLIPS	71
6.3.	Evaluación de la interpretación semántica	71
6.3.1.	Exactitud de objetivos	71

6.3.2.	Validez estructural	71
6.3.3.	Exactitud de entidades y parámetros	71
6.3.4.	Dependencias entre objetivos	71
6.4.	Evaluación de la planificación	71
6.4.1.	Acciones válidas	71
6.4.2.	Consistencia del plan	71
6.4.3.	Cumplimiento de subtareas	71
6.4.4.	Precondiciones y dependencias	71
6.4.5.	Planificabilidad	71
6.5.	Clasificación de errores	71
6.5.1.	Errores de interpretación	71
6.5.2.	Errores de planificación	71
6.5.3.	Errores combinados	71
6.6.	Evaluación física con Justina	71
6.6.1.	Selección de tareas	71
6.6.2.	Procedimiento de ejecución	71
6.6.3.	Criterios de éxito y fallo	71
6.7.	Rendimiento computacional	71
6.7.1.	Latencia	71
6.7.2.	Uso de recursos	71
6.7.3.	Tokens y coste del modelo GPT	71
7.	Resultados	72
7.1.	Resultados de la especialización de Qwen	73
7.1.1.	Comportamiento durante entrenamiento	73
7.1.2.	Evaluación del modelo seleccionado	73
7.2.	Resultados de interpretación semántica	73
7.2.1.	Qwen	73
7.2.2.	GPT	73
7.2.3.	Comparación	73
7.3.	Resultados de planificación	73
7.3.1.	Qwen–CLIPS	73
7.3.2.	GPT directo	73
7.3.3.	Condición de control CLIPS	73
7.3.4.	Comparación	73

7.4.	Análisis de errores	73
7.4.1.	Errores de interpretación	73
7.4.2.	Errores de planificación	73
7.4.3.	Casos representativos	73
7.5.	Resultados de ejecución física	73
7.6.	Rendimiento computacional	73
7.6.1.	Latencia	73
7.6.2.	Recursos computacionales	73
7.6.3.	Tokens y coste	73
8.	Discusión	74
8.1.	Interpretación de los resultados respecto a la hipótesis	75
8.2.	Interpretación semántica	75
8.2.1.	Especialización de Qwen	75
8.2.2.	Capacidad de GPT	75
8.2.3.	Efecto de la variabilidad lingüística	75
8.3.	Planificación de acciones	75
8.3.1.	Expansión determinista mediante CLIPS	75
8.3.2.	Planificación generativa mediante GPT	75
8.3.3.	Separación entre errores de interpretación y planificación	75
8.4.	Comparación Qwen-CLIPS frente a GPT directo	75
8.4.1.	Exactitud	75
8.4.2.	Consistencia	75
8.4.3.	Latencia	75
8.4.4.	Recursos y coste	75
8.4.5.	Dependencia de infraestructura externa	75
8.5.	Generalización fuera de las plantillas GPSR	75
8.6.	Limitaciones y amenazas a la validez	75
8.6.1.	Dominio de tareas	75
8.6.2.	Dataset	75
8.6.3.	Modelo Qwen	75
8.6.4.	Modelo GPT	75
8.6.5.	Plataforma física	75
8.7.	Implicaciones para la planificación en robots de servicio	75

9. Conclusiones y trabajo futuro	76
9.1. Conclusiones principales	76
9.2. Cumplimiento de los objetivos	76
9.3. Evaluación de la hipótesis	76
9.4. Contribuciones de la investigación	76
9.4.1. Representación canónica de órdenes	76
9.4.2. Arquitectura Qwen–CLIPS	76
9.4.3. Comparación con planificación directa mediante GPT	76
9.4.4. Metodología para separar errores de interpretación y planificación	76
9.5. Trabajo futuro	76
10. Conclusiones y Trabajo Futuro	77
10.1. Conclusiones Principales	77
10.2. Propuestas de Trabajo Futuro	77

Índice de figuras

3.1. Arquitectura general ViRBot del robot Justina.	22
3.2. Arquitecturas Qwen-CLIPS.	46
3.3. Arquitectura GPT directo.	50
3.4. Distribución conceptual de las funciones de interpretación y planificación en las arquitecturas Qwen-CLIPS y GPT directo.	55

Capítulo 1

Introducción

Los robots de servicio representan un campo de proyección para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1), donde deben interpretar instrucciones expresadas por personas y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción.(2).

La transformación de una instrucción en lenguaje natural a un plan robótico presenta dos problemas relacionados. El primero es semántico: identificar correctamente los objetivos, entidades, destinos y relaciones expresados por el usuario. El segundo es simbólico: convertir dicha interpretación en una secuencia de acciones pertenecientes al repertorio real del robot y con parámetros compatibles con el ejecutor.

Los sistemas basados en reglas, han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). En particular, CLIPS utiliza hechos y reglas de producción para determinar qué transformaciones deben aplicarse a partir de un estado simbólico. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística puede producir formatos inválidos, entidades inexistentes o acciones que no pertenecen a las capacidades del robot. Por lo tanto la falta de garantías formales de seguridad limitan

su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis estudia dos aproximaciones para resolver el problema. La primera es una arquitectura neuro-simbólica en la que un modelo Qwen2.5-7B, ajustado mediante QLoRA, transforma una orden en objetivos canónicos representados en JSON. Posteriormente, un módulo determinista normaliza y convierte esos objetivos en hechos, que son procesados por reglas de producción en CLIPS para generar el plan compatible con el ejecutor del robot. La segunda aproximación emplea un modelo GPT como planificador generativo directo. En este caso, el modelo recibe la orden, el contexto y el catálogo de acciones, produciendo una respuesta estructurada mediante JSON Schema que es revisada por un validador determinista antes de considerarse ejecutable.

Las dos aproximaciones no se combinan mediante un selector de planes durante la ejecución. Se implementan como arquitecturas independientes y se comparan experimentalmente bajo el mismo conjunto de órdenes, contexto y criterios de evaluación. Esta comparación permite analizar el compromiso entre interpretación semántica, consistencia, trazabilidad, flexibilidad, latencia, recursos computacionales y coste de uso.

1.1. Contexto y Motivación

El área de la robotica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1).

El robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra módulos de percepción, navegación, manipulación e interacción mediante ROS 2. Para ejecutar una orden, el robot necesita una representación intermedia que conecte el lenguaje del usuario con las acciones ofrecidas por dichos módulos.

Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas (*Rule-Based Systems, RBS*), que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, una solución exclusivamente

simbólica puede generar planes consistentes cuando recibe objetivos correctamente estructurados, pero no resuelve directamente la interpretación del lenguaje natural. En contraste, un modelo de lenguaje puede interpretar distintas formulaciones de una orden, aunque su salida no es necesariamente compatible con las restricciones del robot. La motivación de este trabajo es evaluar si la separación entre interpretación neuronal y expansión simbólica permite obtener planes ejecutables y trazables, y contrastar sus resultados con los de un planificador basado exclusivamente en un modelo GPT.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se busca identificar que arquitectura presenta ventajas dependiendo del comando, así como que errores introduce cada etapa y cuáles son los costes asociados con su despliegue local o mediante una API externa.

1.2. Planteamiento del problema

Una orden puede contener variaciones lingüísticas, referencias espaciales, objetos, personas, destinos y varias subtarefas relacionadas. Para que el robot pueda ejecutarla, la orden debe convertirse en una secuencia válida de acciones primitivas, para esto el sistema basados en reglas CLIPS, fundamentado en la lógica simbólica y el ciclo *reconocer-actuar* (3), usa un conjunto de hechos y reglas bien definidos para producir una respuesta determinista y explicable, lo que lo hace idóneo para aplicaciones donde la seguridad es crítica, como entornos industriales o médicos.

La conversión por parte de CLIPS enfrenta los siguientes problemas:

1. **Rigidez interpretativa** No se pueden procesar comandos en lenguaje natural complejo o ambiguo.
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los resultados deben satisfacer al menos las siguientes condiciones (6):

- Las entidades finales deben pertenecer al catálogo o ser tratadas explícitamente como desconocidas.

- Los objetivos deben conservar el significado y el orden causal de la instrucción.
- Las acciones generadas deben pertenecer al repertorio del ejecutor.
- Cada acción debe contener los parámetros requeridos.
- La secuencia debe respetar precondiciones simbólicas básicas, como la ubicación del robot o la posesión de un objeto.

Los modelos de lenguaje pueden ampliar la cobertura lingüística, aunque también pueden generar resultados inexistentes o inconsistentes. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Qué diferencias de exactitud, consistencia, robustez y coste existen entre una arquitectura neuro-simbólica Qwen-CLIPS y un planificador generativo basado en GPT al convertir órdenes GPSR en planes para un robot de servicio?

1.3. Hipótesis

Para la interpretación y planeación de órdenes, una arquitectura neuro-simbólica compuesta por Qwen y CLIPS producirá una mayor proporción de planes estructuralmente válidos, trazables y consistentes que un planificador generativo basado directamente en GPT. Por otra parte, se espera que el planificador GPT directo presente mayor flexibilidad ante formulaciones lingüísticas novedosas, pero con mayor variabilidad, latencia y coste de inferencia mediante API.

Se espera que los sistemas:

- Mejore la tasa de éxito en la ejecución de tareas en entornos domésticos.
- Reduzca el tiempo de respuesta ante comandos complejos.
- Mantenga un comportamiento seguro y predecible en situaciones críticas.
- Permita una interacción más natural e intuitiva con usuarios no expertos.

La hipótesis se evaluará de forma separada para la interpretación de objetivos y para la generación del plan completo. De esta manera será posible determinar si un error proviene del modelo de lenguaje, de la conversión simbólica o de la cobertura de las reglas CLIPS.

1.4. Objetivos

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con un planificador generativo basado en GPT con salida estructurada.

■ Objetivos específicos

- Definir una representación canónica de objetivos y su correspondencia con las acciones disponibles en el ejecutor.
- Ajustar Qwen2.5-7B mediante QLoRA para convertir órdenes en objetivos canónicos representados en JSON.
- Implementar la normalización, validación y conversión de los objetivos generados por Qwen en hechos.
- Implementar reglas de producción en CLIPS que expandan objetivos de alto nivel en secuencias de acciones compatibles con el ejecutor.
- Diseñar un planificador GPT directo mediante un prompt, un JSON Schema y un validador determinista.
- Construir un conjunto experimental que incluya órdenes conocidas, paráfrasis, composiciones de objetivos, entradas ambiguas y casos fuera del dominio.
- Comparar las arquitecturas en términos de exactitud semántica, validez estructural, robustez, latencia, consumo de recursos, variabilidad y coste.
- Validar en simulación, robot físico y durante la RoboCup 2026 el resultado los planes generados.

■ Contribuciones esperadas

- Una representación intermedia de objetivos que conecta la interpretación lingüística con la planeación simbólica.
- Un modelo Qwen2.5-7B ajustado mediante QLoRA para traducir órdenes a objetivos canónicos.
- Una integración ROS 2–CLIPS que convierte dichos objetivos en planes compatibles con el ejecutor.
- Un planificador GPT directo restringido mediante prompt, JSON Schema y validación determinista.
- Un protocolo experimental reproducible para comparar planeación neuro-simbólica y planeación generativa.
- Un análisis de errores por etapa que diferencia fallos semánticos, estructurales, simbólicos y de ejecución.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y validación del sistema híbrido de planificación. La estructura es la siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.
3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología.** Detalla el diseño del sistema híbrido, incluyendo la definición de reglas en CLIPS, la integración con ChatGPT/Qwen, y los mecanismos de planificación dual y validación.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Análisis de resultados.** Compara el rendimiento del sistema híbrido frente al sistema basado solo en reglas, evalúa la efectividad de los LLMs y discute limitaciones y costos computacionales.
8. **Capítulo 8: Discusión.** Interpreta los resultados en relación con los objetivos, analiza ventajas y desventajas del enfoque híbrido, y explora implicaciones éticas y de seguridad.
9. **Capítulo 9: Contribuciones y relevancia.** Destaca la aportación técnica del marco híbrido, su aplicabilidad práctica en distintos sectores y su relevancia académica como benchmark.
10. **Capítulo 10: Conclusiones y trabajo futuro.** Resume los hallazgos principales y propone líneas de investigación futura, como el fine-tuning de LLMs y la

mejora de mecanismos de seguridad.

Capítulo 2

Antecedentes y Estado del Arte

Este capítulo presenta los trabajos y tecnologías relacionados con la transformación de instrucciones de lenguaje natural a planes para robots de servicio. La revisión se concentra en cuatro componentes relevantes para la tesis: sistemas basados en reglas, CLIPS, modelos de lenguaje para interpretación y planificación, y arquitecturas neuro-simbólicas. Los métodos de percepción, navegación o control se mencionan únicamente cuando contribuyen directamente a la interfaz entre la orden y el plan de tareas.

2.1. Sistemas Basados en Reglas

Los sistemas basados en reglas (*Rule-Based Systems, RBS*) constituyen uno de los paradigmas clásicos de la inteligencia artificial (3). Su utilización se consolidó con el desarrollo de los sistemas expertos durante las décadas de 1970 y 1980, en los que el conocimiento de especialistas humanos era representado explícitamente mediante hechos y reglas dentro de dominios bien delimitados (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales estructurados, donde la previsibilidad y seguridad eran prioritarias.

Este tipo de representación resulta útil cuando las condiciones bajo las cuales deben ejecutarse determinadas acciones pueden expresarse de manera explícita. Las reglas permiten relacionar estados, eventos u objetivos con decisiones específicas, facilitando la construcción de comportamientos reproducibles dentro de dominios previamente definidos. Entre las herramientas desarrolladas para implementar este paradigma se encuentra CLIPS (*C Language Integrated Production System*), desarrollado originalmente por la NASA. CLIPS proporciona mecanismos para representar conocimiento mediante

hechos y reglas de producción, junto con un motor de inferencia encargado de determinar qué reglas deben activarse de acuerdo con el estado de la base de conocimiento (7).

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** Cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** En tareas simples como la entrega de objetos, navegación en interiores y asistencia a personas con movilidad reducida.
3. **Robótica espacial y de exploración:** Sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).

Su principal ventaja para esta tesis es que permiten establecer una correspondencia explícita entre un objetivo simbólico y las acciones emitidas para el robot. Sin embargo, la cobertura del sistema está limitada por las reglas y los tipos de hechos definidos previamente.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en (*LISP*, *LISt Processing*) en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS es un entorno para el desarrollo de sistemas basados en conocimiento que ofrece hechos, plantillas, reglas de producción, funciones y un motor de inferencia que permite razonar sobre una base de conocimiento mediante reglas de producción (3).

En el contexto de esta tesis, las propiedades relevantes de CLIPS son:

1. **Encadenamiento hacia adelante (*forward chaining*):** CLIPS utiliza predominantemente encadenamiento hacia adelante. A partir de los hechos presentes en la base de conocimiento, el motor identifica las reglas cuyas condiciones se satisfacen y las incorpora a la agenda de ejecución. La ejecución de estas reglas puede modificar posteriormente la base de hechos y habilitar nuevas reglas (7).
2. **Representación explícita del conocimiento:** Además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas (*deftemplate*), así como mediante funciones definidas por el usuario en C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (9).
3. **Trazabilidad de las reglas activadas:** Debido a que las decisiones dependen de hechos y reglas explícitas, es posible inspeccionar las reglas que son activadas durante una ejecución. Cuando se mantienen constantes los hechos iniciales, las reglas y la estrategia de resolución de conflictos, el comportamiento del motor puede reproducirse de manera determinista.
4. **Portabilidad:** CLIPS fue implementado en C con el propósito de facilitar su utilización en diferentes plataformas. Esta característica permite incorporar el motor de inferencia dentro de sistemas de software más amplios sin depender de una plataforma especializada (3,7).
5. **Integración con otros lenguajes:** CLIPS proporciona interfaces que permiten incorporarlo dentro de aplicaciones desarrolladas en otros lenguajes. En el caso de Python, esta comunicación puede realizarse mediante bindings externos que exponen las funciones del motor de CLIPS. Esto permite utilizar el sistema basado en reglas como un componente dentro de una arquitectura robótica más amplia.

Estas propiedades contribuyen a restringir y validar el comportamiento del sistema, pero no implican, por sí mismas, una garantía de seguridad ni constituyen un mecanismo de verificación formal.

CLIPS tampoco interpreta directamente lenguaje natural ni aprende reglas a partir de los ejemplos empleados en esta investigación. Para recibir una orden requiere una etapa externa que produzca objetivos estructurados. Además, la integración con ROS

2 no es una capacidad intrínseca de CLIPS, sino una interfaz implementada específicamente para comunicar el motor de reglas con los nodos del robot.

2.3. Modelos de lenguaje para interpretación de instrucciones robóticas

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha ampliado las capacidades de los sistemas robóticos para procesar información compleja y entradas con un alto grado de variabilidad (10). A diferencia de los sistemas basados en reglas, en los que parte del conocimiento del dominio debe representarse explícitamente, los enfoques de aprendizaje automático permiten obtener representaciones y patrones a partir de datos (2).

En el procesamiento de información secuencial, arquitecturas como las redes neuronales recurrentes (RNN), LSTM y GRU permitieron modelar dependencias entre los elementos de una secuencia y fueron utilizadas en diferentes tareas relacionadas con el procesamiento del lenguaje natural (10).

Posteriormente, la arquitectura Transformer introdujo un enfoque basado principalmente en mecanismos de atención, permitiendo modelar relaciones entre los elementos de una secuencia sin depender exclusivamente de recurrencia (5). Esta arquitectura constituye la base de gran parte de los modelos de lenguaje de gran escala desarrollados posteriormente.

Los *Large Language Models* (LLMs) basados en Transformers pueden procesar y generar lenguaje natural utilizando el contexto de una instrucción para producir diferentes tipos de representaciones. En robótica, estos modelos se han utilizado para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas e incluso generar planes a partir de descripciones de tareas de alto nivel (6).

Estas capacidades resultan relevantes para esta investigación debido a que permiten utilizar un modelo de lenguaje como interfaz entre las expresiones proporcionadas por el usuario y las representaciones estructuradas utilizadas posteriormente por los componentes de planificación.

2.3.1. Familia de modelos Qwen

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. La familia incluye modelos de diferentes tamaños y variantes destinadas a tareas generales y al seguimiento de instrucciones (13). Estos modelos están basados en la arquitectura Transformer y pueden utilizarse en tareas como generación de texto, respuesta a preguntas, extracción de información y producción de contenido estructurado.

Qwen2.5 corresponde a una generación de esta familia que incluye modelos con diferentes números de parámetros. Para esta investigación se utiliza **Qwen/Qwen2.5-7B** como componente neuronal encargado de la interpretación de las instrucciones proporcionadas al robot. Los criterios que motivaron la selección de esta variante se desarrollan posteriormente.

En la arquitectura propuesta, Qwen2.5-7B funciona como intérprete semántico y no como generador del plan final. El modelo recibe una orden expresada en lenguaje natural y produce una secuencia de objetivos canónicos representada en formato JSON. Esta secuencia identifica los tipos de objetivo, las entidades, las ubicaciones y las relaciones extraídas de la instrucción. Posteriormente, un módulo determinista normaliza y valida la respuesta antes de convertirla en hechos que pueden ser procesados por CLIPS.

De esta manera, el modelo de lenguaje se utiliza específicamente para reducir la variabilidad lingüística presente en las instrucciones, mientras que la generación de las acciones necesarias para alcanzar los objetivos permanece bajo responsabilidad del componente simbólico.

Justificación de la selección de Qwen2.5-7B

La selección de Qwen2.5-7B como modelo base responde tanto a las características requeridas por la tarea como a las restricciones computacionales disponibles durante el desarrollo de esta investigación. La tarea planteada requiere interpretar instrucciones expresadas en lenguaje natural y transformarlas en una representación estructurada de objetivos canónicos, por lo que resulta conveniente utilizar un modelo con capacidad suficiente para seguir instrucciones, conservar las relaciones semánticas presentes en un orden y generar salidas con una estructura consistente.

Durante las etapas preliminares de desarrollo se realizaron pruebas con Qwen2.5-0.5B debido a sus menores requerimientos computacionales y a la posibilidad de efectuar rápidamente iteraciones de ajuste y evaluación. Estas pruebas tuvieron un carácter

exploratorio y permitieron validar inicialmente el flujo de entrenamiento y la representación utilizada para los objetivos.

Sin embargo, durante estas pruebas se observó que las respuestas del modelo presentaban una consistencia insuficiente para los requerimientos de la tarea, particularmente ante instrucciones con mayor complejidad o variabilidad lingüística. Debido a ello, se optó por utilizar una variante con mayor capacidad para los experimentos posteriores.

Estas observaciones iniciales se utilizaron como criterio de diseño para seleccionar la siguiente variante del modelo y no constituyen, por sí mismas, una comparación exhaustiva entre los diferentes tamaños de la familia Qwen2.5.

Se seleccionó finalmente Qwen2.5-7B como un compromiso entre capacidad del modelo y viabilidad computacional. Bajo la configuración de entrenamiento utilizada y los recursos computacionales disponibles durante esta investigación, la variante de siete mil millones de parámetros representó el mayor tamaño que resultó práctico para realizar de manera local los procesos de ajuste y experimentación. La plataforma utilizada estuvo compuesta por una GPU NVIDIA GeForce RTX 4070 Ti, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM.

La utilización de modelos de mayor tamaño habría incrementado los requerimientos de memoria y procesamiento, además de reducir la flexibilidad para realizar múltiples iteraciones experimentales dentro de la infraestructura disponible. Por esta razón, se optó por mantener el modelo dentro de un tamaño que permitiera realizar tanto su adaptación como su evaluación localmente.

La elección de Qwen2.5-7B no pretende establecer que este modelo sea superior de manera general a otras familias de modelos de lenguaje. Existen diversas alternativas que podrían emplearse para una tarea de este tipo; sin embargo, la selección se realizó considerando el equilibrio entre capacidad para procesar instrucciones, disponibilidad del modelo, posibilidad de ejecutarlo y adaptarlo localmente, y los recursos computacionales disponibles. De esta manera, Qwen2.5-7B constituye una solución viable para evaluar la integración de un modelo de lenguaje especializado dentro de la arquitectura neuro-simbólica propuesta.

2.3.2. Modelos GPT

Los modelos GPT constituyen otra familia de modelos generativos basados en la arquitectura Transformer. En esta tesis, el modelo seleccionado se utiliza mediante la API de OpenAI, en lugar de desplegar sus parámetros dentro de la infraestructura local.

La API recibe la instrucción y el contexto proporcionado para la tarea y devuelve una respuesta generada por el modelo.

En el capítulo de implementación se especifican el modelo y la configuración utilizados durante los experimentos, debido a que una denominación general como “ChatGPT” no identifica de manera reproducible un modelo específico accesible mediante API (14).

En la arquitectura experimental GPT directo, el modelo recibe la orden, el catálogo de entidades, el repertorio de acciones, las restricciones del dominio y ejemplos de respuesta. A diferencia de Qwen implementado en la arquitectura neuro-simbólica, este modelo genera tanto la representación de los objetivos como la secuencia propuesta de acciones. Por ello, constituye un planificador generativo independiente y no un módulo interno de la cadena Qwen-CLIPS.

La respuesta del modelo se solicita mediante una salida estructurada definida a partir de JSON Schema. La documentación de la API denomina *Structured Outputs* al mecanismo mediante el cual la respuesta generada debe ajustarse al esquema proporcionado (14). Este mecanismo facilita la integración de la salida con los componentes posteriores del sistema, aunque el cumplimiento del esquema no garantiza por sí mismo la validez semántica del contenido generado.

Los distintos niveles de validación empleados para las salidas de los modelos se describen posteriormente en la subsección de salidas estructuradas y validación.

El uso de una API externa introduce variables que deben incluirse en la evaluación, como latencia de comunicación, disponibilidad del servicio, tokens de entrada y salida, coste monetario y posibles cambios entre versiones del modelo. En consecuencia, las pruebas deben fijar y documentar el modelo, los parámetros de generación y el prompt.

2.3.3. Interpretación semántica y generación de planes robóticos

Para esta tesis conviene distinguir dos funciones:

Interpretación semántica: Transformación de una orden en una representación intermedia de objetivos, entidades y relaciones.

Planificación generativa: Producción directa de una secuencia de acciones a partir de la orden, el contexto y las capacidades declaradas del robot.

En la arquitectura Qwen-CLIPS estas funciones se mantienen separadas: Qwen realiza la interpretación semántica de la orden y CLIPS expande los objetivos obtenidos

en una secuencia de acciones. En la arquitectura GPT directo, ambas funciones se concentran en un único modelo, que produce tanto la representación de los objetivos como el plan propuesto. Esta distinción permite comparar el efecto de mantener una etapa simbólica explícita frente a delegar conjuntamente la interpretación y la composición del plan a un modelo generativo.

La interpretación semántica puede evaluarse comparando la secuencia de objetivos producida por el modelo con una anotación de referencia. La planificación requiere criterios adicionales: pertenencia de las acciones al repertorio del robot, validez de parámetros, conservación de dependencias, cumplimiento de precondiciones y compatibilidad con el ejecutor. Evaluar ambas etapas de forma separada permite localizar el origen de los errores.

2.3.4. Limitaciones de los modelos de lenguaje en robótica

El uso de un modelo de lenguaje introduce riesgos específicos: generación de entidades o acciones inexistentes, incumplimiento del formato, variabilidad entre ejecuciones, sensibilidad al *prompt* y costes de inferencia. Asimismo, una instrucción ambigua puede dar lugar a una interpretación plausible pero diferente de la intención del usuario.

Estas limitaciones son especialmente relevantes cuando la salida se utiliza para controlar un sistema físico. Por ello, una respuesta lingüísticamente coherente no debe considerarse automáticamente un plan ejecutable. En esta investigación se separan la generación, la validación estructural y la validación semántica; los módulos de ejecución del robot conservan la responsabilidad sobre restricciones físicas y condiciones observadas durante la tarea.

2.3.5. Salidas estructuradas y validación

Una estrategia para integrar modelos de lenguaje con software convencional consiste en restringir la salida a una estructura legible por máquina. JSON permite representar objetivos, parámetros y acciones, mientras que JSON Schema permite comprobar tipos, campos obligatorios, enumeraciones y restricciones estructurales.

No obstante, una respuesta conforme al esquema puede seguir siendo semánticamente incorrecta. Por ejemplo, puede incluir una ubicación que no pertenece al mapa o aplicar una acción válida al tipo de entidad equivocado. En consecuencia, la validación debe separar al menos tres niveles:

1. Sintaxis JSON.
2. Conformidad con el esquema.
3. Validez semántica respecto a catálogos, acciones y precondiciones.

Esta separación es aplicable tanto a los objetivos producidos por Qwen como al plan generado directamente por GPT.

2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA

El ajuste completo de un modelo de lenguaje requiere actualizar todos sus parámetros y mantener estados adicionales durante el entrenamiento. Los métodos de adaptación eficiente en parámetros permiten reducir este coste al entrenar únicamente un subconjunto reducido de parámetros, manteniendo congelada gran parte del modelo preentrenado (15, 16). LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas del modelo, manteniendo congelados los pesos originales (16), mientras que QLoRA combina estos adaptadores con la cuantización del modelo base para reducir significativamente el uso de memoria durante el ajuste fino (17).

En esta investigación se utiliza QLoRA para especializar Qwen2.5-7B en la conversión de órdenes a objetivos canónicos. Los detalles del conjunto de datos, cuantización, hiperparámetros y plataforma pertenecen al capítulo de implementación.

2.4. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas buscan combinar las capacidades de aprendizaje y generalización de los modelos neuronales con mecanismos explícitos de representación de conocimiento y razonamiento simbólico. Mientras que los modelos neuronales son especialmente adecuados para procesar información no estructurada y aprender representaciones a partir de datos, los sistemas simbólicos permiten expresar conocimiento mediante estructuras discretas, reglas y relaciones que pueden ser utilizadas durante un proceso de inferencia (18, 19). El objetivo de este enfoque es aprovechar las fortalezas de ambos paradigmas en lugar de utilizar cada uno de manera aislada.

La integración entre ambos componentes puede realizarse de diferentes maneras. Un modelo neuronal puede, por ejemplo, transformar una entrada no estructurada en una representación simbólica que posteriormente es procesada por un sistema de razonamiento. En la dirección opuesta, el conocimiento simbólico puede utilizarse para

imponer restricciones, validar resultados o proporcionar información adicional durante el aprendizaje o la inferencia del modelo neuronal (19). Por tanto, una arquitectura neuro-simbólica no implica necesariamente que el aprendizaje neuronal y el razonamiento simbólico se realicen dentro de un único modelo.

Esta separación resulta particularmente útil cuando la entrada del sistema se encuentra expresada en lenguaje natural, pero la ejecución posterior requiere representaciones estructuradas y operaciones sujetas a reglas explícitas. En este tipo de arquitecturas, el modelo neuronal puede encargarse de interpretar la variabilidad lingüística y transformarla en una representación formal o canónica, mientras que un componente simbólico utiliza dicha representación para realizar razonamiento o planificación.

Un ejemplo de integración entre modelos de lenguaje y planificación simbólica es la arquitectura LLM+P, propuesta en *Empowering Large Language Models with Optimal Planning Proficiency* (20). En este enfoque, el modelo de lenguaje no se utiliza directamente como el mecanismo encargado de buscar la secuencia final de acciones. En su lugar, participa en la transformación de una descripción expresada en lenguaje natural hacia una representación formal que posteriormente puede ser procesada por un planificador clásico.

Para esta representación se utiliza PDDL (*Planning Domain Definition Language*), un lenguaje destinado a describir problemas de planificación automática. De manera general, PDDL permite representar los elementos que caracterizan al dominio de planificación, como las acciones disponibles, sus parámetros, precondiciones y efectos, así como los elementos específicos de un problema, entre ellos los objetos involucrados, el estado inicial y el objetivo que debe alcanzarse.

Una vez obtenida la representación formal del problema, la búsqueda de una secuencia de acciones que satisfaga el objetivo se delega a un planificador clásico. De esta manera, el modelo de lenguaje participa principalmente en la interpretación y formalización de la descripción proporcionada, mientras que el razonamiento necesario para obtener el plan se realiza mediante un mecanismo simbólico.

De manera simplificada, este flujo puede representarse como:

Lenguaje natural $\rightarrow$ Modelo de lenguaje $\rightarrow$ PDDL $\rightarrow$ Planificador clásico $\rightarrow$ Plan.

Esta separación permite asignar a cada componente una función distinta: el modelo de lenguaje se encarga principalmente de interpretar expresiones lingüísticas y convertirlas

en una representación estructurada, mientras que el planificador trabaja sobre dicha representación para determinar una secuencia de acciones que satisfaga las restricciones del problema (20). Este enfoque constituye un ejemplo de cómo un componente neuronal puede utilizarse como interfaz entre lenguaje natural y un mecanismo simbólico de planificación.

La arquitectura principal de esta tesis sigue una idea conceptualmente relacionada, aunque no utiliza PDDL ni reproduce directamente la arquitectura LLM+P. En este trabajo, el modelo Qwen2.5 (21), adaptado al dominio de tareas de servicio del robot, recibe una instrucción expresada en lenguaje natural y la transforma en una representación canónica de los objetivos solicitados. Esta representación constituye la interfaz entre el componente neuronal y el componente simbólico de la arquitectura.

El propósito de esta primera etapa es abstraer parte de la variabilidad presente en el lenguaje natural, como diferentes estructuras gramaticales, sinónimos o formas alternativas de expresar una misma intención. De este modo, distintas instrucciones que representan una intención equivalente pueden transformarse en objetivos con una estructura común antes de ser procesadas por el sistema de planificación.

En la segunda etapa, los objetivos obtenidos son enviados a CLIPS. A diferencia del modelo neuronal, CLIPS no interpreta directamente la instrucción original en lenguaje natural. Su función consiste en trabajar sobre los símbolos previamente obtenidos, incorporar los objetivos al estado de conocimiento y aplicar las reglas definidas para descomponerlos y expandirlos en las acciones necesarias para su ejecución. De esta manera, la interpretación lingüística y la expansión simbólica del plan permanecen como responsabilidades diferenciadas dentro de la arquitectura.

Por tanto, aunque LLM+P y la arquitectura propuesta en esta tesis utilizan mecanismos y representaciones diferentes, ambas ilustran una separación entre interpretación neuronal y razonamiento simbólico. En LLM+P, el modelo de lenguaje genera una representación PDDL que posteriormente es procesada por un planificador clásico; en esta tesis, Qwen genera objetivos canónicos que son posteriormente procesados mediante reglas de producción en CLIPS. La semejanza se encuentra, por tanto, en la separación funcional entre ambos paradigmas y no en el formalismo simbólico empleado.

Esta división permite que el modelo neuronal se concentre en el problema de correspondencia entre lenguaje natural y representaciones canónicas, mientras que el conocimiento sobre la estructura de las tareas, sus relaciones y su descomposición se mantiene explícitamente en la base de reglas de CLIPS. En consecuencia, una modificación en las

reglas de planificación no requiere necesariamente modificar el modelo de lenguaje y, de forma análoga, las mejoras realizadas sobre el componente de interpretación pueden introducirse sin sustituir el mecanismo simbólico de planificación.

En esta tesis, por tanto, la integración neuro-simbólica se describe como una composición secuencial: Qwen transforma el lenguaje natural en objetivos canónicos y CLIPS expande dichos objetivos mediante reglas de producción. Los componentes no actúan como dos planificadores concurrentes ni se realiza una fusión automática de planes generados independientemente. El denominado planificador GPT directo se implementa como una arquitectura alternativa con fines de comparación experimental y no como un componente adicional de la cadena neuro-simbólica Qwen–CLIPS.

2.5. Síntesis y conexión con la propuesta

Los antecedentes revisados muestran que los sistemas simbólicos permiten mantener una representación explícita del conocimiento utilizado durante la planificación, mientras que los modelos de lenguaje ofrecen mecanismos para procesar la variabilidad presente en las instrucciones expresadas en lenguaje natural. En una arquitectura híbrida, la combinación de ambos enfoques permite separar el problema de interpretación lingüística del proceso encargado de expandir los objetivos en acciones ejecutables.

Esta tesis estudia dicha separación mediante una comparación controlada entre una arquitectura secuencial Qwen–CLIPS y un planificador GPT directo. Adicionalmente, se utiliza una condición experimental de control en la que los objetivos de referencia son proporcionados directamente a CLIPS. Esta condición permite observar el comportamiento de la etapa simbólica cuando los objetivos de entrada corresponden directamente a la referencia, evitando introducir los errores provenientes de la etapa de interpretación del lenguaje natural. De esta manera, es posible distinguir los errores asociados con la interpretación lingüística de aquellos relacionados con la cobertura y expansión de las reglas de planificación. Esta condición no constituye una tercera arquitectura de planificación, sino un mecanismo auxiliar para aislar los errores correspondientes a la etapa simbólica.

El modelo Qwen2.5-7B ajustado mediante QLoRA recibe una instrucción y produce una secuencia de objetivos canónicos representada en JSON. Un módulo determinista valida y normaliza la respuesta, la convierte en hechos y la envía al motor CLIPS. Finalmente, las reglas de producción expanden cada objetivo en acciones compatibles

con el ejecutor del robot.

Como segunda arquitectura evaluada, se implementa un planificador basado en GPT que genera conjuntamente una representación estructurada de los objetivos interpretados y del plan propuesto. Su salida se restringe mediante JSON Schema y posteriormente se somete a validación determinista. Ambas arquitecturas se evalúan de forma independiente; por tanto, el trabajo no propone una selección concurrente ni una fusión automática de los planes generados.

La contribución experimental consiste en comparar ambos enfoques bajo el mismo conjunto de órdenes, catálogos y acciones, separando los errores de interpretación de los errores de planificación y cuantificando su desempeño mediante métricas de exactitud, ejecutabilidad, consistencia, latencia, uso de recursos y coste.

Capítulo 3

Formalización del problema y arquitectura conceptual

El problema abordado en esta tesis consiste en transformar una instrucción expresada en lenguaje natural en una secuencia de acciones que pueda ser procesada por el sistema de ejecución de un robot de servicio. Esta transformación requiere relacionar dos niveles de representación diferentes: por una parte, la intención expresada por el usuario y, por otra, las acciones concretas disponibles en el robot.

Como se revisó en el capítulo anterior, los modelos de lenguaje proporcionan mecanismos para procesar la variabilidad presente en las instrucciones lingüísticas, mientras que los sistemas simbólicos permiten representar conocimiento y relaciones mediante estructuras explícitas. A partir de estos antecedentes, en este capítulo se formaliza el problema particular considerado en la investigación y se establecen las representaciones utilizadas para describir órdenes, objetivos y planes.

El dominio experimental toma como referencia las tareas de General Purpose Service Robot (GPSR) de RoboCup@Home. Sin embargo, GPSR se utiliza como una fuente controlada de tareas, entidades y escenarios, y no como una restricción que obligue al usuario a reproducir literalmente las formulaciones lingüísticas utilizadas en la competencia.

Finalmente, se presentan las dos arquitecturas que serán evaluadas. La primera utiliza Qwen2.5-7B como intérprete semántico y CLIPS como componente encargado de expandir los objetivos obtenidos en acciones ejecutables. La segunda delega tanto la interpretación como la generación del plan a un modelo GPT. Ambas arquitecturas utilizan el mismo dominio conceptual, pero distribuyen de manera diferente la respon-

sabilidad de transformar una orden en un plan.

3.1. Contexto arquitectónico y alcance del sistema

3.1.1. Integración dentro de la arquitectura ViRBot

La investigación se desarrolla sobre el robot de servicio Justina y su arquitectura de software ViRBot. Esta arquitectura integra diferentes capacidades necesarias para la operación del robot, entre ellas percepción, interacción humano-robot, navegación, manipulación, representación de conocimiento, planificación y ejecución.

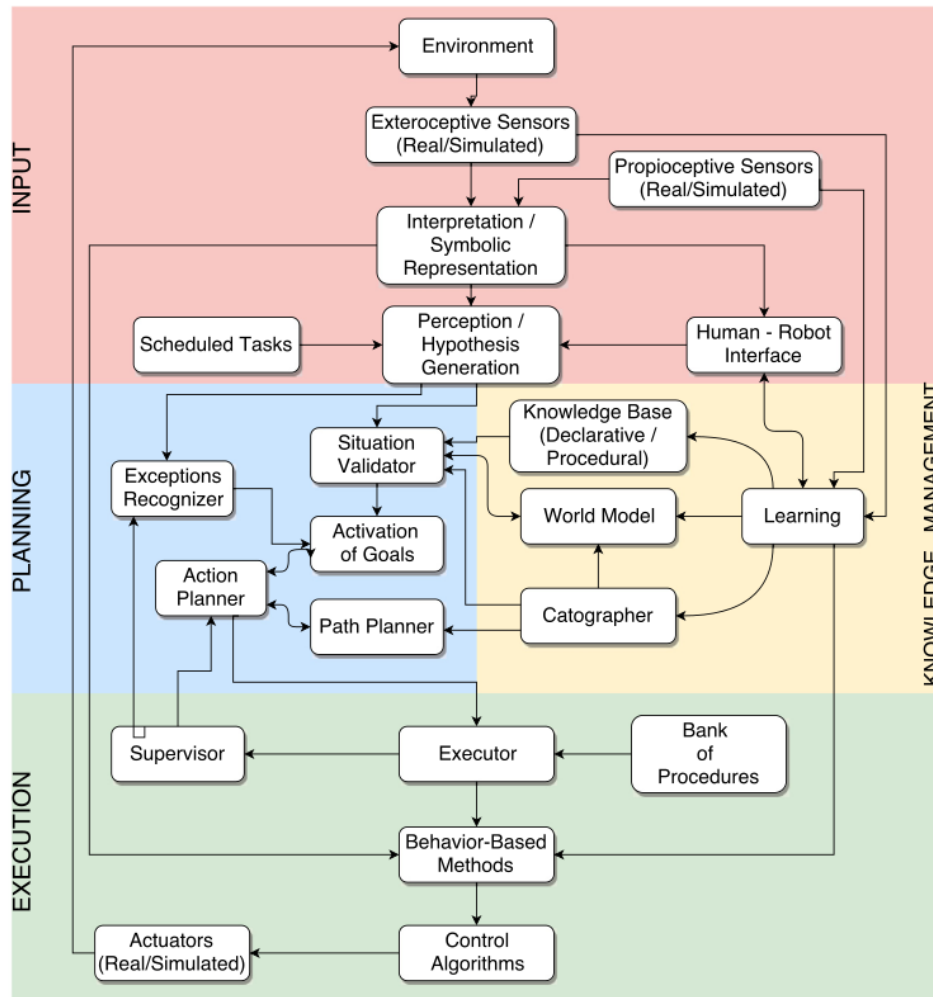


Figura 3.1: Arquitectura general ViRBot del robot Justina.

El sistema desarrollado en esta tesis se ubica entre la recepción de una instrucción

expresada en lenguaje natural y los componentes encargados de ejecutar las acciones correspondientes. Por tanto, no pretende sustituir los módulos de navegación, percepción o manipulación del robot, sino proporcionar una representación ordenada de las acciones que dichos módulos deben realizar.

De manera simplificada, el flujo general puede expresarse como:

$$\boxed{\text{orden en lenguaje natural} \rightarrow \text{interpretación y planificación} \rightarrow \text{plan} \rightarrow \text{ejecución}} \quad (3.1)$$

La contribución estudiada se concentra principalmente en las transformaciones anteriores a la ejecución física. Los módulos existentes de Justina son los responsables de intentar realizar las acciones especificadas en el plan.

3.1.2. Alcance del problema abordado

El trabajo se concentra en la transformación de instrucciones en lenguaje natural en planes de acciones compatibles con las capacidades del robot.

Una instrucción proporcionada por una persona puede contener diferentes subtareas relacionadas entre sí. Por ejemplo:

“Go to the kitchen, find an apple and bring it to me.”

Esta instrucción contiene información relacionada con navegación, búsqueda, manipulación y entrega. Para que pueda ser procesada por el robot es necesario identificar dichas intenciones y convertirlas posteriormente en operaciones compatibles con sus capacidades.

Los módulos se consideran componentes externos al problema de planificación estudiado y se utilizan posteriormente para realizar las acciones producidas.

3.2. GPSR como dominio de referencia

3.2.1. La prueba General Purpose Service Robot

General Purpose Service Robot (GPSR) es una prueba de RoboCup@Home orientada a evaluar la capacidad de un robot de servicio para comprender instrucciones expresadas en lenguaje natural y ejecutar tareas que pueden requerir la combinación

de distintas habilidades. Entre ellas se encuentran la navegación, la búsqueda y reconocimiento de personas u objetos, la manipulación y la interacción humano-robot(?).

Una característica importante de GPSR es que una instrucción puede representar una tarea compuesta. Por ejemplo, una orden puede requerir que el robot se desplace hacia una habitación, localice un objeto, lo tome y posteriormente lo entregue a una persona. En consecuencia, el problema no consiste únicamente en reconocer palabras o entidades aisladas, sino en identificar las diferentes intenciones contenidas en la instrucción y conservar las relaciones existentes entre ellas.

En la prueba considerada en esta investigación se proporcionan tres comandos al robot. Al comenzar, el robot se desplaza hacia un *Instruction Point*, que funciona como el punto de interacción con el operador. Desde esta posición recibe una instrucción y, una vez que ha realizado suficientemente la tarea solicitada, regresa al mismo punto para recibir el siguiente comando. La determinación de que una tarea ha sido completada de manera suficiente para continuar con la prueba corresponde al árbitro.

La entrega de las órdenes se realiza de manera secuencial. Por tanto, el robot no recibe un nuevo comando mientras la ejecución de la tarea anterior continúe en curso. Una vez concluida la tarea y después de regresar al *Instruction Point*, puede solicitar la siguiente instrucción hasta completar los tres comandos contemplados por la prueba.

Cuando el robot no logra comprender adecuadamente la instrucción recibida mediante su sistema de entrada de audio, puede solicitar explícitamente que la orden sea repetida. En este caso, el árbitro u operador expresara la misma tarea mediante una construcción lingüística más sencilla, pero manteniendo los objetivos contenidos en la instrucción original. La reformulación no constituye una modificación de la tarea ni permite que el operador indique directamente al robot cómo resolverla.

Este mecanismo es particularmente relevante para la presente investigación, pues permite distinguir entre la tarea que debe realizarse y la forma lingüística utilizada para comunicarla. Dos instrucciones con estructuras sintácticas diferentes pueden representar los mismos objetivos y, por tanto, deberían conducir a una interpretación semántica equivalente.

Durante la ejecución no se contempla que una persona intervenga para descomponer la orden, seleccionar las acciones del robot o corregir manualmente el plan generado. La interpretación de la instrucción y la determinación de las acciones necesarias corresponden al propio sistema robótico. De esta manera, GPSR proporciona un escenario apropiado para evaluar la transformación de instrucciones en lenguaje natural en re-

presentaciones semánticas y planes de acción.

3.2.2. Generación de comandos y conocimiento del dominio

Una característica relevante de GPSR es que los comandos utilizados durante la prueba no se seleccionan únicamente a partir de una lista fija de oraciones. El reglamento establece que las tareas se construyen mediante el *CommandGenerator* oficial de RoboCup@Home (?). Sobre esta herramienta se realizaron posteriormente adaptaciones específicas para construir el conjunto de datos empleado en el entrenamiento y evaluación del modelo. Dichas modificaciones se describen en el capítulo de metodología.

El generador define diferentes familias de instrucciones relacionadas con personas y objetos. Entre ellas se encuentran tareas para desplazarse hacia una ubicación, encontrar personas u objetos, contar entidades, obtener información, seguir o guiar personas, tomar objetos y entregarlos o colocarlos en una ubicación determinada. Algunas de estas estructuras permiten incorporar continuaciones que producen comandos compuestos. De esta manera, una instrucción inicial puede extenderse con subtareas posteriores que dependen del resultado de las anteriores (?).

De forma simplificada, el proceso puede representarse como:

$$\boxed{\text{familia de tarea} + \text{estructura lingüística} + \text{entidades del dominio} \rightarrow \text{comando GPSR}} \quad (3.2)$$

Las estructuras del generador utilizan elementos variables para representar entidades y relaciones del dominio. Por ejemplo, una plantilla puede contener una habitación, una persona, un objeto o una ubicación que posteriormente son sustituidos por valores concretos. De forma similar, diferentes verbos pueden representar una intención relacionada; por ejemplo, una tarea de navegación puede expresarse mediante verbos como *go* o *navigate*, mientras que una búsqueda puede utilizar expresiones como *find*, *locate* o *look for* (?).

El conocimiento utilizado para realizar estas sustituciones se mantiene separado de las estructuras de los comandos. El generador distingue, entre otros elementos, nombres de personas, habitaciones, ubicaciones, superficies de colocación, objetos y categorías de objetos (?). La herramienta permite recibir este conocimiento desde un directorio externo y señala el repositorio *CompetitionTemplate* como referencia para su organización (?).

En *CompetitionTemplate*, la información específica de una competencia se organiza en archivos relacionados con el mapa y las ubicaciones, los nombres de personas y los objetos disponibles (?). Esta separación resulta importante porque permite mantener las estructuras generales de las tareas mientras se modifican las entidades concretas presentes en una edición o escenario determinado.

Conceptualmente, puede representarse el dominio de referencia como:

$$D_{\text{GPSR}} = (T, K), \quad (3.3)$$

donde T representa las familias de tareas contempladas por el generador y K el conocimiento específico utilizado para instanciarlas. La estructura detallada de K y su utilización en esta investigación se formalizan en la Sección 3.3.3.

3.2.3. Dominio de referencia y alcance lingüístico

La utilización de GPSR como referencia no implica que el sistema desarrollado en esta tesis deba reconocer únicamente las frases producidas literalmente por el generador oficial. Esta distinción resulta especialmente importante debido a que una misma intención puede expresarse mediante diferentes construcciones lingüísticas.

El propio procedimiento de GPSR 2026 introduce variación sobre las órdenes generadas. Una vez producido un comando mediante el generador oficial, el reglamento establece que éste se proporciona a un modelo de lenguaje para obtener una formulación similar. Además, una instrucción puede ser reformulada hasta tres veces, utilizando expresiones progresivamente más simples cuando sea necesario (?).

Por ejemplo, el reglamento presenta una transformación equivalente a pasar de una instrucción como:

“Get me a coke from the kitchen.”

a una formulación que explicita sus subtareas, como:

“Go to the kitchen, find a coke, and bring it to me.”

Aunque la estructura superficial de ambas expresiones es diferente, la tarea subyacente permanece esencialmente sin cambios. Esto permite distinguir dos niveles: forma lingüística e intención semántica.

Esta distinción es central para la presente investigación. GPSR proporciona un conjunto controlado de tareas y entidades, pero la entrada lingüística puede presentar diferentes formas de expresar una misma intención. El objetivo del intérprete no consiste, por tanto, en memorizar las plantillas del generador, sino en establecer una correspondencia entre distintas realizaciones lingüísticas y una representación semántica común.

Por ejemplo, considérese:

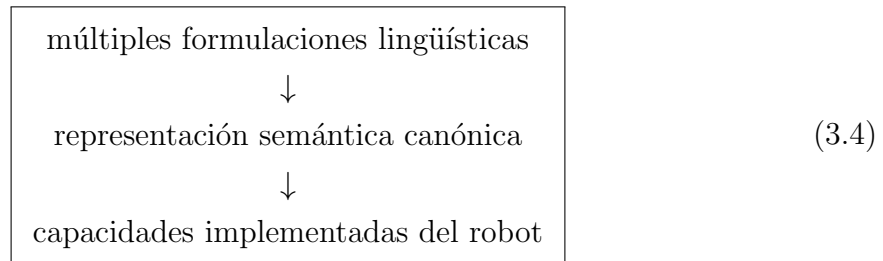
“Go to the kitchen and find Robin.” y “Could you look for Robin in the kitchen?”

Dentro del dominio considerado, ambas pueden corresponder a la misma secuencia de objetivos:

`go(kitchen), find(Robin, kind=person)`

Por tanto, en esta tesis GPSR se utiliza como una fuente controlada para definir las tareas, entidades y relaciones que forman parte del dominio experimental, pero no como una gramática que limite las únicas formulaciones aceptables por el sistema.

Puede resumirse esta relación mediante:



En consecuencia, la entrada lingüística puede variar en su formulación, pero la interpretación permanece acotada por el dominio semántico y las capacidades operativas disponibles. Una instrucción puede ser perfectamente comprensible para una persona y, aun así, quedar fuera del alcance del sistema si requiere una intención que no puede representarse mediante los objetivos contemplados o una capacidad que el robot no posee.

El dominio experimental de esta investigación considera principalmente instrucciones en inglés, en correspondencia con las estructuras utilizadas por el generador y con el escenario de evaluación. Por ello, los resultados relacionados con variabilidad lingüística deben interpretarse como capacidad para procesar diferentes formulaciones de las tareas soportadas dentro de dicho idioma, y no como evidencia de independencia respecto al idioma o de comprensión irrestricta del lenguaje natural.

3.3. Formalización de órdenes, objetivos y planes

La transformación de una instrucción en lenguaje natural a un plan para un robot de servicio implica relacionar representaciones con diferentes niveles de abstracción. La expresión proporcionada por el usuario describe una tarea mediante lenguaje natural, mientras que el sistema de ejecución requiere una secuencia de acciones pertenecientes a un repertorio previamente definido.

En esta investigación se introduce una representación semántica intermedia que permite separar ambos problemas. De manera general, el proceso se representa como:

$$\boxed{x \rightarrow G \rightarrow P} \quad (3.5)$$

donde x representa la instrucción expresada en lenguaje natural, G una secuencia ordenada de objetivos semánticos y P el plan compuesto por acciones compatibles con las capacidades del robot.

Esta separación permite analizar independientemente dos problemas. El primero consiste en determinar qué solicita el usuario: $x \rightarrow G$, mientras que el segundo consiste en determinar qué acciones son necesarias para realizar los objetivos obtenidos: $G \rightarrow P$.

Las siguientes subsecciones formalizan las representaciones utilizadas para describir estas transformaciones.

3.3.1. Orden en lenguaje natural

Sea: $x \in \mathcal{X}$ una instrucción expresada en lenguaje natural perteneciente al conjunto $\mathcal{X}$ de órdenes consideradas dentro del dominio experimental.

Una instrucción puede describir una tarea simple o una composición de varias sub-tareas relacionadas. Por ejemplo:

“Go to the kitchen, find an apple and bring it to me.”

contiene información relacionada con navegación, búsqueda, manipulación y entrega de un objeto.

Por tanto, una instrucción lingüística no corresponde necesariamente con una única acción del robot: $\boxed{\text{una instrucción} \neq \text{una acción}}$

Incluso una expresión lingüísticamente breve puede implicar varias operaciones. Por ejemplo:

“Bring me an apple from the kitchen.”

puede representarse mediante la siguiente secuencia semántica:

`go(kitchen), find(apple, kind=object), take(apple), deliver(apple, to=me)`

En consecuencia, el objetivo de la interpretación no consiste únicamente en identificar palabras clave, sino en determinar las subtareas implícitas o explícitas en la instrucción, las entidades involucradas y las relaciones existentes entre ellas.

Formalmente, la interpretación puede expresarse mediante una función: $I : \mathcal{X} \rightarrow \mathcal{G}^*$, donde $\mathcal{G}^*$ representa el conjunto de secuencias finitas que pueden formarse a partir del vocabulario de objetivos semánticos $\mathcal{G}$.

Para una instrucción particular x , la función produce: $I(x) = G = (g_1, g_2, \dots, g_n)$. El número n de objetivos obtenidos depende de la tarea representada por la instrucción y no necesariamente de su longitud superficial.

3.3.2. Equivalencia semántica entre formulaciones

Como se describió en la Sección 3.2, una misma tarea puede ser expresada mediante diferentes formulaciones lingüísticas. Esta propiedad es particularmente relevante en GPSR, donde una orden puede ser reformulada manteniendo los objetivos de la instrucción original.

Sean dos instrucciones:

$$x_1, x_2 \in \mathcal{X}. \quad (3.6)$$

En el contexto de esta investigación, se consideran semánticamente equivalentes cuando ambas representan la misma secuencia canónica de objetivos. Esta relación se denota mediante:

$$x_1 \sim x_2 \quad (3.7)$$

si:

$$I(x_1) = I(x_2) = G. \quad (3.8)$$

Por ejemplo, las instrucciones:

“Bring me an apple from the kitchen.”

y:

“Go to the kitchen, find an apple, take it and bring it to me.”

presentan estructuras lingüísticas diferentes, pero pueden corresponder a la misma secuencia semántica:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

Por tanto, ambas expresiones pertenecen conceptualmente a una misma clase de equivalencia:

$$[x] = \{x' \in \mathcal{X} \mid I(x') = I(x)\}. \quad (3.9)$$

Esta representación permite expresar uno de los objetivos principales de la etapa de interpretación: diferentes realizaciones superficiales de una misma tarea deben converger hacia una representación semántica común.

De manera esquemática:

$$\boxed{\begin{array}{ccc} x_1 & & \\ x_2 & \longrightarrow & G \\ \vdots & & \\ x_k & & \end{array}} \quad (3.10)$$

donde $x_1, \dots, x_k$ representan distintas formas de expresar una intención equivalente.

Esta propiedad permite separar la variabilidad lingüística del dominio semántico. El sistema puede admitir diferentes formas de formular una tarea sin que ello implique incrementar indefinidamente el conjunto de objetivos que debe ser capaz de representar.

En consecuencia, el problema de interpretación puede entenderse como una transformación desde un espacio lingüístico de alta variabilidad hacia un espacio semántico más reducido y estructurado:

$$\boxed{\mathcal{X} \xrightarrow{I} \mathcal{G}^*} \quad (3.11)$$

El objetivo no es memorizar las estructuras superficiales utilizadas para generar los comandos de referencia, sino identificar la intención que dichas estructuras representan.

3.3.3. Entidades y conocimiento del dominio

La interpretación de una instrucción requiere disponer de información sobre las entidades que pueden participar en las tareas consideradas.

En esta investigación, el conocimiento relevante del dominio se representa como:

$$K = \{K_P, K_O, K_C, K_L, K_R\}, \quad (3.12)$$

donde:

- K_P representa el conjunto de personas o nombres conocidos;
- K_O representa el conjunto de objetos;
- K_C representa las categorías de objetos;
- K_L representa las ubicaciones específicas;
- K_R representa las habitaciones.

Cada uno de estos subconjuntos puede expresarse, de manera general, como:

$$K_P = \{p_1, p_2, \dots, p_{n_P}\}, \quad (3.13)$$

$$K_O = \{o_1, o_2, \dots, o_{n_O}\}, \quad (3.14)$$

y de forma equivalente para los demás componentes del dominio.

Esta información permite determinar el tipo semántico de diferentes referencias. Por ejemplo:

$$Robin \in K_P, \quad (3.15)$$

mientras que:

$$apple \in K_O. \quad (3.16)$$

Por tanto:

`find(Robin, kind=person)`

y:

```
find(apple, kind=object)
```

comparten el mismo tipo general de objetivo, pero operan sobre entidades semánticamente diferentes.

El conocimiento del dominio cumple al menos dos funciones. En primer lugar, permite restringir las entidades nominales que pueden ser consideradas válidas. Una salida que introduzca arbitrariamente una persona, objeto o ubicación que no pertenece al dominio no debe interpretarse automáticamente como correcta.

En segundo lugar, proporciona información para resolver el tipo de las entidades y verificar la compatibilidad entre ellas y los objetivos en los que participan.

Las referencias presentes en una instrucción pueden adoptar distintas formas. En esta investigación se distinguen, de manera conceptual, tres casos principales:

1. **Entidades nominales:** referencias explícitas a una entidad conocida, como `Robin`, `apple` o `kitchen`.
2. **Entidades genéricas:** referencias a una clase de entidad, como `person`, `drink` o `fruit`.
3. **Entidades descritas mediante propiedades:** referencias que requieren identificar una entidad a partir de alguna característica, como una persona saludando o el objeto de mayor tamaño.

Por ejemplo:

```
find(person, kind=person, gesture='waving person')
```

no identifica inicialmente a una persona mediante un nombre, sino mediante una propiedad observable.

De manera similar:

```
find(drink, kind=object, property=largest)
```

representa la búsqueda de un objeto perteneciente a una categoría y sujeto a una propiedad adicional.

Esta distinción permite conservar información semántica que posteriormente puede ser utilizada por los componentes de percepción y planificación.

3.3.4. Representación canónica de objetivos

El resultado de la interpretación se representa mediante una secuencia ordenada:

$$G = (g_1, g_2, \dots, g_n), \quad (3.17)$$

donde cada g_i representa un objetivo semántico.

De manera general:

$$g_i = (t_i, \theta_i), \quad (3.18)$$

donde t_i identifica el tipo de objetivo y θ_i contiene los argumentos y calificadores necesarios para describirlo.

Equivalentemente, puede utilizarse una notación funcional:

$$g_i = t_i(\theta_i). \quad (3.19)$$

El vocabulario semántico utilizado en esta investigación se define como:

$$\begin{aligned} \mathcal{G} = \{ & \text{go, find, take, deliver, place, drop,} \\ & \text{guide, follow, count, tell, talk, save,} \\ & \text{answer_question, greet} \}. \end{aligned} \quad (3.20)$$

La Tabla 3.1 resume el propósito general de cada tipo de objetivo y sus argumentos principales.

Los objetivos **find** y **count** requieren distinguir explícitamente si operan sobre personas u objetos. Para ello se utiliza el argumento **kind**:

```
find(Robin, kind=person)
find(apple, kind=object)
```

```
count(person, kind=person)
count(drinks, kind=object)
```

Otros objetivos poseen un dominio semántico implícito. Por ejemplo, **take**, **deliver**, **place** y **drop** operan sobre objetos, mientras que **guide**, **follow** y **greet** operan sobre personas.

Tabla 3.1: Vocabulario de objetivos semánticos utilizado en la representación canónica.

Objetivo	Propósito	Argumentos principales
go	Desplazarse hacia una ubicación	ubicación
find	Localizar una persona u objeto	objetivo, kind , gesto, pose, vestimenta o propiedad
take	Tomar un objeto	objeto
deliver	Entregar un objeto	objeto, destinatario
place	Colocar un objeto en un destino	objeto, destino y relación espacial
drop	Depositar un objeto	objeto, destino y relación espacial
guide	Guiar a una persona	persona, destino
follow	Seguir a una persona	persona, destino opcional
count	Contar personas u objetos	objetivo, kind y atributos opcionales
tell	Comunicar información	información o propiedad
talk	Producir información verbal	contenido
save	Conservar información obtenida durante una interacción	nombre, pose o gesto
answer_question	Atender una pregunta	—
greet	Saludar a una persona	persona

Los objetivos también pueden conservar calificadores relevantes presentes en la instrucción. Entre ellos se encuentran propiedades relacionadas con gestos, poses, vestimenta, características de objetos y relaciones de destino.

Por ejemplo:

```
find(person, kind=person, gesture='waving person')
```

```
find(person, kind=person, pose='sitting person')
```

```
find(person, kind=person, wearing='red shirt')
```

```
find(drink, kind=object, property=largest)
```

```
guide(Robin, to=kitchen)
```

```
place(apple, on=table)
```

La secuencia completa se serializa posteriormente mediante una estructura JSON. Por ejemplo:

```
{
  "goals": [
    "go(living room)",
    "find(Robin, kind=person)",
    "follow(Robin, to=kitchen)"
  ]
}
```

El formato de serialización no modifica la naturaleza conceptual de la representación. El elemento relevante para la formalización es la secuencia G , mientras que JSON constituye el mecanismo utilizado para intercambiar dicha información entre componentes.

Fidelidad de la representación semántica

Una secuencia compuesta únicamente por objetivos válidos no constituye necesariamente una interpretación correcta de una instrucción.

La representación canónica debe conservar las intenciones relevantes de la orden y eliminar únicamente la variabilidad superficial que no modifica su significado dentro del dominio.

Considérese la instrucción:

“Go to the kitchen and find Robin.”

La secuencia esperada es:

```
go(kitchen)
find(Robin, kind=person)
```

Una salida como:

```
find(Robin, kind=person)
```

utiliza un objetivo válido, pero omite una subtarea solicitada explícitamente por la orden.

De manera similar:

```

go(kitchen)
find(Robin, kind=person)
greet(Robin)

```

incorpora una intención adicional que no forma parte de la instrucción original.

Por tanto, una interpretación correcta debe satisfacer, conceptualmente, las siguientes propiedades:

- preservar las subtareas solicitadas;
- no introducir objetivos ajenos a la instrucción;
- conservar las entidades relevantes;
- conservar las relaciones entre dichas entidades;
- mantener las restricciones de orden necesarias para completar la tarea.

Si se dispone de una secuencia de referencia G_{ref} , una condición ideal de interpretación puede expresarse como:

$$G_{\text{pred}} = G_{\text{ref}}, \quad (3.21)$$

donde G_{pred} representa la interpretación producida por el modelo.

Esta igualdad constituye una condición estricta y posteriormente puede descomponerse en métricas más específicas para analizar qué componentes de la interpretación fueron correctos o incorrectos.

3.3.5. Objetivos semánticos y acciones ejecutables

Los objetivos canónicos describen **qué** debe conseguir el robot, pero no necesariamente especifican directamente todas las operaciones requeridas para conseguirlo.

Sea $\mathcal{G}$ el vocabulario de objetivos semánticos definido anteriormente y sea:

$$\mathcal{A} \quad (3.22)$$

el repertorio de acciones disponibles para el sistema de ejecución.

En esta investigación:

$$\boxed{\mathcal{G} \neq \mathcal{A}.} \quad (3.23)$$

Entre las acciones utilizadas por el ejecutor se encuentran, por ejemplo:

$$\mathcal{A} \supseteq \{\text{go_to}, \text{say}, \text{listen}, \text{find_person}, \text{find_pose}, \\ \text{approach}, \text{focus}, \text{find_object}, \text{analyze_objects}, \\ \text{analyze_person}, \text{take}, \text{find_space}, \text{drop}, \text{give}\}. \quad (3.24)$$

Algunos elementos pueden poseer nombres similares en ambos vocabularios, como **take**, pero esto no implica que ambas representaciones sean equivalentes en general.

Por ejemplo, el objetivo semántico:

`deliver(apple, to=Robin)`

representa una intención de alto nivel. Su realización puede requerir varias operaciones, como localizar al destinatario, aproximarse y ejecutar la entrega mediante la acción correspondiente.

Conceptualmente:

$$\text{deliver}(o, p) \rightarrow (\text{find_person}, \text{approach}, \text{give}, \dots). \quad (3.25)$$

De manera similar:

`place(apple, on=table)`

puede requerir identificar un espacio adecuado antes de ejecutar la acción de liberar el objeto:

$$\text{place}(o, l) \rightarrow (\text{find_space}, \text{drop}, \dots). \quad (3.26)$$

La secuencia completa de acciones se define como:

$$P = (a_1, a_2, \dots, a_m), \quad (3.27)$$

donde:

$$a_i \in \mathcal{A}. \quad (3.28)$$

Cada objetivo puede producir una subsecuencia de acciones:

$$g_i \rightarrow (a_j, a_{j+1}, \dots, a_k). \quad (3.29)$$

Por tanto, el problema de planificación considerado puede expresarse de manera general como una transformación:

$$R : \mathcal{G}^* \rightarrow \mathcal{A}^*, \quad (3.30)$$

de forma que:

$$R(G) = P. \quad (3.31)$$

Esta distinción es fundamental para las arquitecturas comparadas en esta investigación. La etapa de interpretación produce elementos pertenecientes a $\mathcal{G}$, mientras que el resultado final de planificación debe estar compuesto por elementos pertenecientes a $\mathcal{A}$.

3.3.6. Dependencias y contexto entre objetivos

La secuencia G no puede tratarse únicamente como un conjunto independiente de objetivos. Algunas subtareas presentan relaciones de precedencia, referencias comparadas o condiciones que dependen del resultado de objetivos anteriores.

Por este motivo:

$$G = (g_1, g_2, \dots, g_n) \quad (3.32)$$

se considera una **secuencia ordenada de objetivos** y no simplemente un conjunto:

$$G \neq \{g_1, g_2, \dots, g_n\}. \quad (3.33)$$

Dependencias de precedencia

Algunos objetivos requieren que otra subtask haya sido realizada previamente.

Por ejemplo:

```
take(apple)
deliver(apple, to=Robin)
```

requiere que el objeto haya sido adquirido antes de intentar entregarlo.

Esta relación puede representarse como:

$$\text{take}(o) \prec \text{deliver}(o, p), \quad (3.34)$$

donde $\prec$ representa una relación de precedencia.

De manera equivalente:

$$\text{take}(o) \prec \text{place}(o, l). \quad (3.35)$$

Una secuencia que invirtiera estas relaciones podría contener objetivos individualmente válidos, pero no representaría una composición coherente de la tarea.

Dependencias referenciales

Algunas instrucciones introducen entidades que posteriormente deben ser reutilizadas.

Considérese:

```
find(person, kind=person, gesture='waving person')
guide(person, to=kitchen)
```

El argumento **person** utilizado en el segundo objetivo no representa una persona arbitraria, sino la entidad identificada como resultado del objetivo anterior.

Conceptualmente, esta relación puede expresarse como:

$$\text{ref}(g_j) = \text{result}(g_i), \quad i < j, \quad (3.36)$$

donde $\text{result}(g_i)$ representa la entidad obtenida como resultado de una subtarea anterior.

Estas relaciones permiten representar expresiones lingüísticas que utilizan pronombres, referencias genéricas o descripciones cuyo referente se determina durante la ejecución de la tarea.

Dependencias espaciales

La interpretación de una subtarea también puede depender del contexto espacial establecido por objetivos anteriores.

Por ejemplo:

```
go(kitchen)
find(apple, kind=object)
```

establece que la búsqueda del objeto debe realizarse después de desplazarse hacia la cocina.

En este caso, el primer objetivo establece el contexto espacial esperado para la ejecución del segundo.

De manera conceptual, puede considerarse que determinados objetivos producen una actualización del estado esperado:

$$S_{i+1} = T(S_i, g_i), \quad (3.37)$$

donde S_i representa un estado abstracto antes de procesar el objetivo g_i y T una función conceptual de transición.

Así, un objetivo:

$$\text{go}(l) \quad (3.38)$$

puede producir conceptualmente:

$$\text{location}(S_{i+1}) = l. \quad (3.39)$$

Esta representación no implica que el movimiento físico haya sido ejecutado con éxito; únicamente representa el estado esperado utilizado durante la composición del plan.

Dependencias de estado

Algunas subtarefas modifican condiciones que son necesarias para objetivos posteriores.

Por ejemplo:

```
take(apple)
deliver(apple, to=Robin)
```

establece conceptualmente un estado intermedio en el que el robot posee el objeto:

$$\text{holding}(S_{i+1}) = \text{apple}. \quad (3.40)$$

El objetivo posterior de entrega presupone esta condición.

De esta forma, las relaciones entre objetivos no dependen exclusivamente de su posición dentro de la secuencia, sino también del estado esperado producido por las subtareas anteriores.

Orden lingüístico y orden semántico

Finalmente, debe distinguirse entre el orden superficial en que aparecen los elementos dentro de una oración y el orden lógico requerido para realizar la tarea.

Por ejemplo:

“Bring me the apple from the kitchen.”

expresa directamente la intención de entrega, pero para realizarla es necesario construir una secuencia semántica como:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

Por tanto, la interpretación no consiste en copiar el orden de aparición de las palabras, sino en recuperar las relaciones necesarias entre las subtareas.

Para este ejemplo:

$$\text{go} \prec \text{find} \prec \text{take} \prec \text{deliver}. \quad (3.41)$$

Esta propiedad resulta especialmente importante en instrucciones compactas, donde algunas operaciones necesarias para cumplir la intención pueden no aparecer expresadas como verbos independientes.

En consecuencia, la representación semántica G debe conservar tanto la intención de la orden como las relaciones necesarias para que sus objetivos puedan ser posteriormente transformados en un plan coherente.

La formalización desarrollada en esta sección permite resumir el problema considerado mediante:

$$\boxed{x \xrightarrow{I} G \xrightarrow{R} P} \quad (3.42)$$

con:

$$x \in \mathcal{X}, \quad G \in \mathcal{G}^*, \quad P \in \mathcal{A}^*. \quad (3.43)$$

La transformación I abstrae la variabilidad del lenguaje natural y conserva la intención mediante una representación canónica, mientras que R transforma dicha representación en una secuencia de acciones pertenecientes al repertorio del robot. Las dos arquitecturas estudiadas en esta tesis se diferencian principalmente en la manera en que implementan estas transformaciones, como se describe en la sección siguiente.

3.4. Arquitecturas conceptuales evaluadas

A partir de la formalización presentada en la Sección 3.3, el problema estudiado puede expresarse mediante dos transformaciones principales:

$$x \rightarrow G \quad (3.44)$$

y:

$$G \rightarrow P, \quad (3.45)$$

donde x representa una instrucción en lenguaje natural, G una secuencia ordenada de objetivos semánticos y P una secuencia de acciones pertenecientes al repertorio ejecutable del robot.

Las dos arquitecturas consideradas en esta investigación resuelven el mismo problema general y utilizan un dominio conceptual común, pero distribuyen de manera diferente la responsabilidad de realizar estas transformaciones. En la arquitectura Qwen-CLIPS, la interpretación lingüística y la expansión del plan se encuentran separadas entre un componente neuronal y uno simbólico. En la arquitectura GPT directo, ambas funciones se concentran en un mismo modelo de lenguaje.

Esta diferencia constituye el eje principal de la comparación experimental. El objetivo no consiste únicamente en comparar dos modelos de lenguaje, sino en analizar dos estrategias distintas para relacionar lenguaje natural, objetivos semánticos y acciones ejecutables.

3.4.1. Base común de las arquitecturas evaluadas

Para que la comparación sea significativa, ambas arquitecturas se consideran sobre una misma definición del problema. En particular, comparten el dominio de conocimiento K , el vocabulario de objetivos semánticos $\mathcal{G}$ y el repertorio de acciones ejecutables $\mathcal{A}$ definidos previamente.

De manera general:

$$K = \{K_P, K_O, K_C, K_L, K_R\}, \quad (3.46)$$

mientras que:

$$G \in \mathcal{G}^* \quad (3.47)$$

y:

$$P \in \mathcal{A}^*. \quad (3.48)$$

En consecuencia, las dos arquitecturas deben interpretar instrucciones dentro del mismo dominio semántico y producir planes compatibles con las mismas capacidades del robot.

La comparación utiliza además un mismo conjunto de órdenes de evaluación y una misma representación de referencia para los objetivos. Esto permite contrastar los resultados sin modificar la definición de las tareas entre una arquitectura y otra.

Sin embargo, compartir el mismo problema no implica que todos los componentes reciban exactamente la misma información ni que el conocimiento necesario para resolverlo se encuentre ubicado en el mismo lugar.

En la arquitectura Qwen-CLIPS, Qwen únicamente debe resolver la transformación:

$$x \rightarrow G. \quad (3.49)$$

El conocimiento necesario para transformar posteriormente esos objetivos en acciones se encuentra representado de forma explícita en el componente simbólico.

Por el contrario, el planificador GPT directo debe producir tanto la interpretación como el plan:

$$x \rightarrow (G, P). \quad (3.50)$$

Por ello, además del conocimiento necesario para interpretar las entidades y objetivos, esta arquitectura debe disponer de información suficiente acerca de las acciones disponibles y de las relaciones necesarias para componer un plan.

Esta diferencia puede resumirse mediante:

$$\boxed{\text{mismo dominio y problema} \not\Rightarrow \text{misma distribución del conocimiento}} \quad (3.51)$$

La comparación se establece, por tanto, sobre el resultado funcional esperado: interpretar correctamente la orden y producir un plan compatible con las capacidades definidas para el robot.

3.4.2. Arquitectura Qwen–CLIPS

La primera arquitectura divide explícitamente el proceso en dos etapas. Qwen se utiliza como intérprete semántico, mientras que CLIPS se encarga de expandir los objetivos obtenidos mediante reglas de producción.

Sea:

$$I_Q : \mathcal{X} \rightarrow \mathcal{G}^* \quad (3.52)$$

la función de interpretación implementada mediante Qwen. Para una instrucción x , el modelo produce una secuencia:

$$G_{\text{pred}} = I_Q(x). \quad (3.53)$$

El modelo no genera directamente el plan final ni selecciona acciones primitivas del robot. Su responsabilidad termina en la identificación de los objetivos semánticos contenidos en la instrucción.

Antes de que la secuencia pueda ser procesada por el componente simbólico, se aplican comprobaciones deterministas destinadas a verificar que la representación producida pertenece al lenguaje canónico definido para el sistema.

De manera conceptual, esta etapa puede representarse mediante:

$$V_G : (G, K) \rightarrow \{\text{válido, inválido}\}. \quad (3.54)$$

Cuando la representación es aceptada, una función de adaptación:

$$C : \mathcal{G}^* \rightarrow \mathcal{F} \quad (3.55)$$

convierte la secuencia de objetivos en una representación simbólica F adecuada para el motor de reglas.

El tipo concreto de hechos utilizado para implementar F se describe en el capítulo de implementación; para la formalización actual basta considerar que esta representación conserva la información contenida en G de una forma que puede ser procesada por CLIPS.

El componente simbólico implementa posteriormente una función:

$$R_C : \mathcal{F} \rightarrow \mathcal{A}^*, \quad (3.56)$$

de manera que:

$$P = R_C(C(G)). \quad (3.57)$$

La arquitectura completa puede representarse entonces como:

$$\boxed{x \xrightarrow{\text{Qwen}} G_{\text{pred}} \xrightarrow{\text{validación}} G' \xrightarrow{C} F \xrightarrow{\text{CLIPS}} P} \quad (3.58)$$

La Figura 3.2 resume el flujo conceptual de la arquitectura Qwen–CLIPS, desde la recepción de la orden en lenguaje natural hasta la generación del plan ejecutable. En ella se observa la separación explícita entre la etapa de interpretación semántica, la validación de los objetivos y su posterior expansión mediante CLIPS.

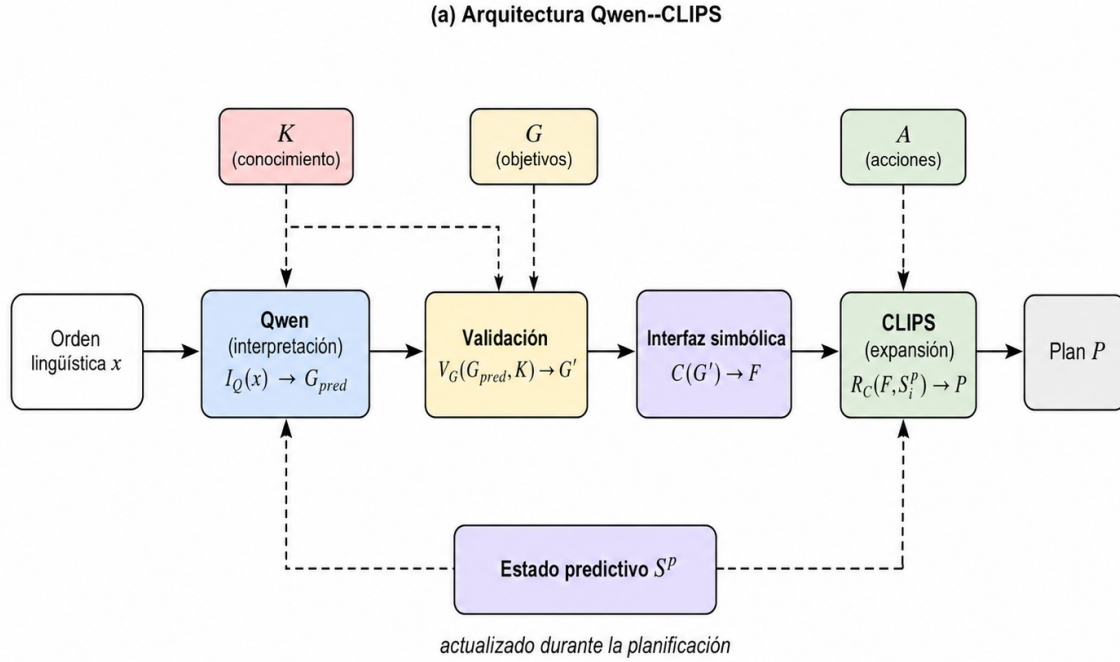


Figura 3.2: Arquitectura conceptual Qwen–CLIPS.

En esta arquitectura, CLIPS funciona como un expansor determinista de objetivos basado en reglas. Su función no consiste en interpretar nuevamente la instrucción original ni en realizar una búsqueda libre entre planes posibles. En cambio, cada objetivo aceptado activa conocimiento explícito que determina las acciones necesarias para atenderlo.

Conceptualmente, un objetivo:

$$g_i \in \mathcal{G} \tag{3.59}$$

puede expandirse en una subsecuencia:

$$g_i \rightarrow (a_j, a_{j+1}, \dots, a_k), \quad a_j, \dots, a_k \in \mathcal{A}. \tag{3.60}$$

Por ejemplo, un objetivo de alto nivel asociado con la entrega de un objeto puede requerir localizar al destinatario, aproximarse y ejecutar posteriormente la acción de

entrega. De forma similar, un objetivo de colocación puede requerir identificar primero una superficie o espacio adecuado.

La expansión de los objetivos conserva el orden de la secuencia semántica obtenida durante la interpretación. Esto permite mantener las relaciones de precedencia y contexto descritas en la Sección 3.3.6.

Estado predictivo durante la expansión

La expansión de cada objetivo no se realiza necesariamente de forma aislada. Algunas decisiones dependen del resultado esperado de objetivos anteriores.

Para representar esta información se considera un estado predictivo:

$$S_i^p, \quad (3.61)$$

donde el superíndice p indica que se trata de un estado utilizado durante la planificación y no de una observación confirmada del mundo físico.

El procesamiento de un objetivo puede representarse conceptualmente como:

$$(g_i, S_i^p) \xrightarrow{R_C} (P_i, S_{i+1}^p), \quad (3.62)$$

donde P_i representa la subsecuencia de acciones producida para atender g_i .

Por ejemplo, después de procesar un objetivo de navegación hacia una ubicación l , el estado predictivo puede asumir:

$$location(S_{i+1}^p) = l. \quad (3.63)$$

De manera semejante, después de procesar un objetivo de adquisición de un objeto o , puede mantenerse conceptualmente:

$$holding(S_{i+1}^p) = o. \quad (3.64)$$

Estas actualizaciones permiten evitar determinadas inconsistencias durante la expansión de objetivos posteriores. Sin embargo, no deben interpretarse como evidencia de que la acción física haya sido completada.

En consecuencia:

$$\boxed{S_i^p \neq S_i^{\text{físico}}} \quad (3.65)$$

en general, ya que la confirmación del estado real corresponde a los componentes encargados de ejecutar y observar las acciones del robot.

Propagación y localización de errores

La separación entre interpretación y expansión permite identificar con mayor claridad el origen de ciertos errores.

Si Qwen produce:

$$G_{\text{pred}} \neq G_{\text{ref}}, \quad (3.66)$$

pero G_{pred} es estructuralmente válida, CLIPS puede generar un plan internamente coherente con dicha interpretación aunque ésta no corresponda con la intención original del usuario.

En ese caso puede ocurrir:

$$x \rightarrow G_{\text{incorrecto}} \rightarrow P_{\text{coherente con } G_{\text{incorrecto}}}. \quad (3.67)$$

Por tanto:

$$\boxed{\text{plan internamente coherente} \not\Rightarrow \text{interpretación correcta}} \quad (3.68)$$

Esta propiedad motiva la evaluación independiente de G y P , así como la utilización posterior de una condición de control en la que los objetivos de referencia se proporcionan directamente al componente CLIPS.

3.4.3. Arquitectura de planificación directa con GPT

La segunda arquitectura concentra en un mismo modelo las funciones de interpretación semántica y generación del plan.

A diferencia de Qwen-CLIPS, el modelo no produce una representación que deba ser enviada posteriormente a un componente simbólico para obtener las acciones. Para una instrucción considerada ejecutable, GPT produce simultáneamente una secuencia de objetivos G y un plan P .

De forma general, se define:

$$D_{\text{GPT}} : (\mathcal{X}, K, \mathcal{G}, \mathcal{A}) \rightarrow \mathcal{Y}, \quad (3.69)$$

donde $\mathcal{Y}$ representa el espacio de respuestas admitidas por la arquitectura.

La salida no se limita al caso en el que siempre sea posible generar un plan. En esta investigación se consideran tres clases conceptuales de respuesta:

$$\boxed{Y = \text{Executable}(G, P) \mid \text{Clarification}(q) \mid \text{Rejected}(r)} \quad (3.70)$$

La primera corresponde a una orden que puede interpretarse y convertirse en un plan dentro del dominio:

$$Y = \text{Executable}(G, P). \quad (3.71)$$

La segunda representa una situación en la que falta información necesaria o existe una ambigüedad que puede resolverse mediante una respuesta breve del usuario:

$$Y = \text{Clarification}(q), \quad (3.72)$$

donde q representa la pregunta necesaria para completar la interpretación.

Finalmente:

$$Y = \text{Rejected}(r) \quad (3.73)$$

representa una instrucción que no puede convertirse de manera válida en un plan dentro del dominio o las capacidades declaradas, y r contiene la razón correspondiente.

Cuando el resultado es ejecutable, el proceso conceptual puede representarse como:

$$\boxed{x \xrightarrow{\text{GPT}} (G_{\text{pred}}, P_{\text{pred}}) \xrightarrow{\text{validación}} (G', P')} \quad (3.74)$$

La Figura 3.3 muestra la distribución conceptual de responsabilidades en la arquitectura GPT directo. A diferencia de Qwen-CLIPS, la interpretación de la orden y la generación del plan se concentran en un mismo modelo, aunque los objetivos y las acciones se conservan como representaciones diferenciadas.

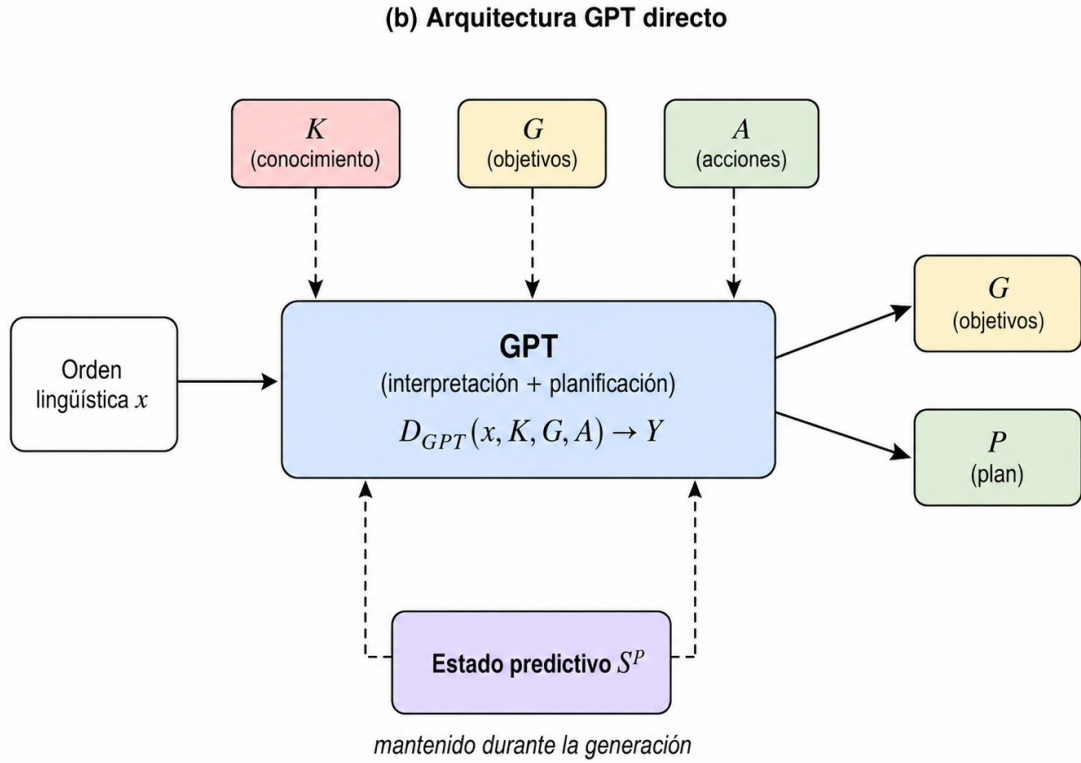


Figura 3.3: Arquitectura GPT directo.

Los objetivos generados por GPT se conservan como una representación explícita de la interpretación realizada por el modelo:

$$G_{\text{GPT}} \in \mathcal{G}^*, \quad (3.75)$$

mientras que el plan debe estar compuesto por acciones pertenecientes al repertorio ejecutable:

$$P_{\text{GPT}} \in \mathcal{A}^*. \quad (3.76)$$

Aunque ambos artefactos se producen durante una misma inferencia, representan niveles de abstracción diferentes y pueden ser evaluados de manera independiente.

Una diferencia fundamental respecto a Qwen-CLIPS es que los objetivos producidos por GPT no son utilizados como entrada para CLIPS:

$$G_{\text{GPT}} \xrightarrow{\text{CLIPS}} P_{\text{GPT}}. \quad (3.77)$$

En cambio, el propio modelo realiza la composición del plan.

Planificación directa bajo restricciones

El término *planificación directa* se utiliza en esta tesis para indicar que el modelo produce las acciones del plan sin delegar la expansión de los objetivos a CLIPS. Esto no implica que la generación se realice sin restricciones.

El modelo opera dentro de un contexto que especifica, entre otros elementos, el conocimiento del dominio K , el vocabulario permitido de objetivos $\mathcal{G}$, el repertorio de acciones $\mathcal{A}$ y las relaciones necesarias para construir planes compatibles con las tareas consideradas.

Por tanto, la generación puede expresarse como:

$$(G, P) = D_{\text{GPT}}(x, K, \mathcal{G}, \mathcal{A}, \mathcal{R}), \quad (3.78)$$

donde $\mathcal{R}$ representa el conjunto de restricciones y relaciones de composición proporcionadas al modelo.

Estas restricciones permiten, por ejemplo, impedir que una intención de alto nivel como `guide` o `deliver` aparezca directamente como una acción ejecutable cuando no pertenece al repertorio $\mathcal{A}$.

De esta forma, GPT debe conservar la distinción previamente establecida entre:

$$\mathcal{G} \quad \text{y} \quad \mathcal{A}. \quad (3.79)$$

El conocimiento que en Qwen-CLIPS se encuentra representado explícitamente mediante reglas de producción debe estar disponible para GPT a través del contexto y las restricciones utilizadas durante la generación.

Estado predictivo durante la generación

La generación directa del plan también requiere conservar coherencia entre las acciones producidas.

Sea:

$$S_0^p \quad (3.80)$$

el estado predictivo inicial utilizado para razonar sobre la tarea. Después de una acción a_i , el estado esperado puede representarse como:

$$S_{i+1}^p = T(S_i^p, a_i). \quad (3.81)$$

Por ejemplo, después de generar una acción destinada a tomar un objeto o , las acciones posteriores pueden considerar conceptualmente:

$$\text{holding}(S_{i+1}^p) = o. \quad (3.82)$$

Esto permite conservar dependencias como:

$$\text{take}(o) \prec \text{give}(o, p). \quad (3.83)$$

Al igual que en la arquitectura Qwen-CLIPS, este estado es predictivo y no representa retroalimentación física proveniente del robot.

Por tanto:

$$\boxed{\text{estado utilizado para generar el plan} \neq \text{confirmación del estado físico}} \quad (3.84)$$

La diferencia se encuentra en el mecanismo que mantiene dicha coherencia: Qwen-CLIPS utiliza representaciones y reglas simbólicas explícitas, mientras que GPT debe conservarla durante la generación condicionada por el contexto proporcionado.

Separación posterior de interpretación y planificación

Aunque GPT realice ambas funciones dentro de una misma inferencia, la preservación explícita de G y P permite separar posteriormente los errores observados.

Si:

$$G_{\text{GPT}} \neq G_{\text{ref}}, \quad (3.85)$$

el error puede asociarse principalmente con la interpretación de la orden.

En cambio, si:

$$G_{\text{GPT}} = G_{\text{ref}} \quad (3.86)$$

pero:

$$P_{\text{GPT}} \quad (3.87)$$

viola las restricciones de la tarea o no constituye una expansión adecuada de los objetivos, puede identificarse un error de planificación.

Esta separación no corresponde a dos módulos internos independientes, como en Qwen–CLIPS, sino a un análisis posterior basado en los dos artefactos producidos por el modelo.

3.4.4. Comparación conceptual de las arquitecturas

Las dos arquitecturas pueden resumirse mediante:

$$\begin{array}{l} \text{Qwen–CLIPS: } x \xrightarrow{\text{Qwen}} G \xrightarrow{\text{CLIPS}} P \\ \text{GPT directo: } x \xrightarrow{\text{GPT}} (G, P) \end{array} \quad (3.88)$$

La diferencia principal no consiste solamente en el modelo de lenguaje utilizado. En Qwen–CLIPS, el conocimiento necesario para interpretar el lenguaje y el conocimiento necesario para expandir las tareas se encuentran separados entre dos componentes:

$$\underbrace{x \rightarrow G}_{\text{interpretación mediante Qwen}} + \underbrace{G \rightarrow P}_{\text{expansión mediante CLIPS}} \quad (3.89)$$

En GPT directo, ambas transformaciones se concentran en un mismo modelo:

$$\underbrace{x \rightarrow (G, P)}_{\text{interpretación y planificación mediante GPT}} \quad (3.90)$$

La Tabla 3.2 resume las principales diferencias conceptuales.

La Figura 3.4 presenta gráficamente esta distribución de responsabilidades.

Debe señalarse que el carácter basado en reglas de CLIPS no implica por sí mismo que cualquier plan producido sea correcto desde el punto de vista físico. La reproducibilidad de su expansión depende de mantener constantes los hechos iniciales, las reglas

Tabla 3.2: Comparación conceptual entre las arquitecturas evaluadas.

Característica	Qwen–CLIPS	GPT directo
Interpretación semántica	Qwen	GPT
Generación del plan	CLIPS	GPT
Representación explícita de G	Sí	Sí
Generación de P	Reglas de producción	Generación condicionada del LLM
Conocimiento de expansión	Representado explícitamente en CLIPS	Proporcionado mediante contexto y restricciones
Separación entre interpretación y planificación	Explícita entre componentes	Integrada en una misma inferencia
Estado durante planificación	Predictivo y simbólico	Predictivo durante la generación
Dependencia de un servicio externo de terceros	No	Sí
Análisis de errores	Separación directa entre etapas	Separación posterior mediante G y P
Variabilidad del resultado	Determinada por los objetivos, reglas, hechos iniciales y estrategia de resolución	Puede presentar variación entre inferencias

y la estrategia de resolución de conflictos. De manera similar, restringir la respuesta de GPT a un vocabulario y una estructura determinados no garantiza que la interpretación o el plan sean semánticamente correctos.

En ambos casos es necesario evaluar por separado la representación de objetivos y el plan resultante.

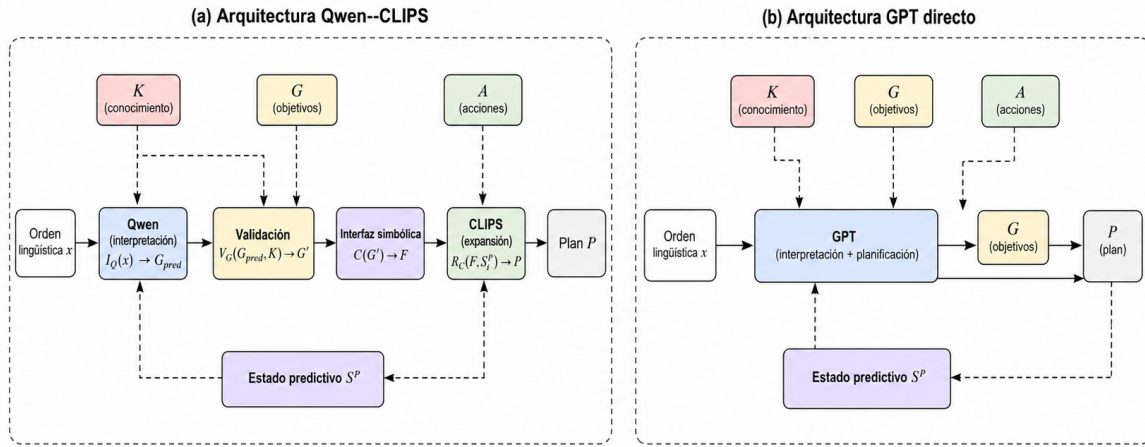
En consecuencia, la comparación propuesta en esta investigación puede interpretarse como el contraste entre dos estrategias:

$$\boxed{\begin{array}{c} \text{separación explícita entre interpretación y planificación} \\ \text{Qwen–CLIPS} \end{array}} \quad (3.91)$$

frente a:

$$\boxed{\begin{array}{c} \text{integración de interpretación y planificación} \\ \text{GPT directo} \end{array}} \quad (3.92)$$

Comparación conceptual de las arquitecturas evaluadas



Distribución de responsabilidades en las arquitecturas Qwen-CLIPS y GPT directo

Figura 3.4: Distribución conceptual de las funciones de interpretación y planificación en las arquitecturas Qwen-CLIPS y GPT directo.

Esta diferencia proporciona la base para analizar posteriormente no sólo qué arquitectura obtiene una mayor proporción de resultados correctos, sino también en qué etapa se originan los errores, qué grado de consistencia presenta cada estrategia y qué recursos requiere para transformar una instrucción en un plan compatible con el robot.

3.5. Criterios conceptuales de validez

La generación de una salida estructurada no implica, por sí misma, que la interpretación de una orden o el plan correspondiente sean correctos. En esta investigación se distinguen varios niveles de validez con el propósito de separar los errores asociados con la representación lingüística, la semántica, la planificación y la ejecución física.

De manera general, el proceso completo puede representarse como:

$$\boxed{x \xrightarrow{V_I} G \xrightarrow{V_P} P \xrightarrow{V_E} E} \quad (3.93)$$

donde x representa la instrucción en lenguaje natural, G la secuencia de objetivos semánticos, P el plan de acciones y E el resultado observado durante la ejecución física.

Las funciones V_I , V_P y V_E representan, respectivamente, la validez de la interpretación, la validez del plan y el resultado de ejecución. Estas dimensiones se analizan de forma diferenciada, ya que el cumplimiento de una de ellas no implica necesariamente el cumplimiento de las siguientes.

3.5.1. Niveles de validez

Para evaluar las salidas producidas por las arquitecturas consideradas, resulta útil distinguir varios niveles conceptuales de validez.

1. **Validez sintáctica:** la salida puede ser interpretada por el software y respeta la sintaxis esperada.
2. **Validez estructural:** la salida utiliza los tipos de datos, campos, objetivos, argumentos y acciones admitidos por el contrato del sistema.
3. **Validez semántica:** la representación obtenida conserva la intención expresada en la instrucción original y utiliza correctamente las entidades y relaciones del dominio.
4. **Validez del plan:** la secuencia de acciones constituye una expansión coherente de los objetivos y respeta las dependencias necesarias para realizar la tarea.
5. **Validez de ejecución:** las acciones planificadas pueden ser realizadas satisfactoriamente por el robot en el entorno físico.

Estos niveles forman una jerarquía conceptual, pero no deben interpretarse como implicaciones automáticas.

Por ejemplo:

$$\boxed{V_{\text{sintáctico}} = 1 \not\Rightarrow V_{\text{semántico}} = 1} \quad (3.94)$$

Una salida puede encontrarse correctamente estructurada y, aun así, contener una entidad incorrecta o representar una intención diferente de la solicitada.

De forma similar:

$$\boxed{V_P = 1 \not\Rightarrow V_E = 1} \quad (3.95)$$

debido a que un plan conceptualmente correcto puede fallar posteriormente durante su ejecución física.

Esta separación permite evitar que diferentes tipos de error sean agrupados bajo una única noción de “plan incorrecto”.

3.5.2. Validez de la interpretación semántica

Sea:

$$V_I(x, G, K) \quad (3.96)$$

una función conceptual que determina si la secuencia de objetivos G representa adecuadamente la instrucción x dentro del conocimiento de dominio K .

De manera simplificada:

$$V_I(x, G, K) \in \{0, 1\}. \quad (3.97)$$

Una interpretación semánticamente válida debe conservar la información relevante de la orden sin introducir intenciones que no hayan sido solicitadas.

Para ello se consideran las siguientes propiedades.

Cobertura de las subtareas

La secuencia G debe contener los objetivos necesarios para representar la tarea expresada por el usuario.

Considérese:

“Go to the kitchen and find Robin.”

La representación:

```
find(Robin, kind=person)
```

contiene un objetivo válido, pero omite la navegación solicitada.

Por tanto, una interpretación puede ser estructuralmente correcta y, aun así, ser semánticamente incompleta.

Ausencia de intenciones no solicitadas

La interpretación tampoco debe agregar objetivos que no formen parte de la orden.

Para la misma instrucción:

```
go(kitchen)
find(Robin, kind=person)
greet(Robin)
```

introduce una tarea de saludo que no fue solicitada.

En consecuencia, la representación canónica debe conservar la intención original y no utilizar la normalización como mecanismo para añadir comportamientos adicionales.

Exactitud de entidades y tipos

Las entidades utilizadas en los objetivos deben corresponder con las presentes en la instrucción y ser compatibles con el conocimiento del dominio.

Si:

$$Robin \in K_P, \quad (3.98)$$

entonces una representación compatible sería:

```
find(Robin, kind=person)
```

y no una clasificación de Robin como objeto.

De forma análoga, las ubicaciones, objetos y categorías utilizadas deben mantener correspondencia con las referencias expresadas en la orden.

Conservación de relaciones y dependencias

Una interpretación válida debe conservar no sólo las entidades individuales, sino también las relaciones existentes entre ellas.

Por ejemplo:

```
find(person, kind=person, gesture='waving person')
guide(person, to=kitchen)
```

requiere que la persona utilizada en el segundo objetivo corresponda con la persona identificada por el primero.

Las dependencias de precedencia descritas anteriormente también forman parte de la interpretación. Una tarea de entrega de un objeto debe conservar la relación entre adquisición y entrega, aun cuando algunas de estas operaciones se encuentren implícitas en la formulación lingüística original.

Consistencia ante reformulaciones

La posibilidad de expresar una misma tarea mediante diferentes formulaciones introduce un criterio adicional de validez.

Como se definió en la Sección 3.3.2, si:

$$x_1 \sim x_2, \quad (3.99)$$

entonces ambas instrucciones representan la misma intención semántica.

Idealmente, el intérprete debe satisfacer:

$$\boxed{x_1 \sim x_2 \Rightarrow I(x_1) = I(x_2)} \quad (3.100)$$

Esta propiedad resulta especialmente relevante para el dominio GPSR, donde una misma tarea puede ser reformulada mediante una expresión lingüística más simple sin modificar los objetivos originales.

Por tanto, la validez de la interpretación no se limita a evaluar una única frase de manera aislada, sino también la capacidad de conservar una representación semántica estable ante formulaciones equivalentes.

3.5.3. Validez y consistencia del plan

Una vez obtenida una secuencia de objetivos G , debe determinarse si el plan P constituye una realización coherente de dichos objetivos.

Sea:

$$V_P(G, P, K, \mathcal{A}) \quad (3.101)$$

una función conceptual de validación del plan.

De manera simplificada:

$$V_P(G, P, K, \mathcal{A}) \in \{0, 1\}. \quad (3.102)$$

La validez del plan depende de varias propiedades.

Pertenencia al repertorio de acciones

Toda acción incluida en el plan debe pertenecer al conjunto de acciones disponibles para el robot:

$$\forall a_i \in P, \quad a_i \in \mathcal{A}. \quad (3.103)$$

La aparición de una acción inexistente o no soportada constituye un error de planificación incluso si la intención general del plan es correcta.

Validez de parámetros

Las acciones también deben utilizar argumentos compatibles con su función.

Por ejemplo, una acción de navegación debe especificar una ubicación válida, mientras que una operación de entrega debe mantener correspondencia con el objeto y el destinatario implicados en la tarea.

Por tanto, la pertenencia de una acción a $\mathcal{A}$ es una condición necesaria, pero no suficiente para determinar su validez.

Conservación de dependencias

Las acciones deben respetar las relaciones necesarias para realizar los objetivos.

Por ejemplo:

$$\text{take}(o) \prec \text{give}(o, p) \quad (3.104)$$

representa una dependencia relacionada con la posesión del objeto.

De manera similar:

$$\text{find_person}(p) \prec \text{approach}(p) \quad (3.105)$$

representa una relación necesaria para aproximarse a una persona que primero debe ser identificada.

Estas restricciones pueden depender tanto del orden de las acciones como del estado predictivo utilizado durante la planificación.

Satisfacción de los objetivos

Además de utilizar acciones válidas, el plan debe proporcionar una realización de los objetivos presentes en G .

Esta relación puede representarse conceptualmente mediante:

$$P \models G, \quad (3.106)$$

donde el símbolo $\models$ indica que la secuencia de acciones de P satisface los objetivos representados en G .

Esta notación se utiliza en esta tesis de manera conceptual y no implica la utilización de un mecanismo de demostración lógica formal.

Un plan puede estar compuesto únicamente por acciones válidas y, aun así, no satisfacer la tarea si omite acciones necesarias para alguno de los objetivos.

Coherencia con el estado predictivo

Las acciones generadas deben ser compatibles con el estado esperado producido por las acciones anteriores.

Sea S_i^p el estado predictivo antes de ejecutar conceptualmente la acción a_i . Una transición se representa como:

$$S_{i+1}^p = T(S_i^p, a_i). \quad (3.107)$$

Por ejemplo, una operación de entrega de un objeto requiere que el estado predictivo contemple previamente que el robot posee dicho objeto.

Una secuencia que contradiga estas condiciones puede contener acciones individualmente válidas, pero no constituir un plan coherente.

3.5.4. Validez de la respuesta del sistema

No todas las instrucciones recibidas deben producir necesariamente un plan.

Esta consideración es particularmente relevante en la arquitectura GPT directo, donde el espacio de respuestas se definió como:

$$Y = \text{Executable}(G, P) \mid \text{Clarification}(q) \mid \text{Rejected}(r). \quad (3.108)$$

Por tanto, también resulta necesario evaluar si el sistema selecciona correctamente el tipo de respuesta correspondiente a una instrucción.

Sea:

$$V_Y(x, Y) \quad (3.109)$$

una función conceptual de validez de la respuesta.

Una respuesta:

$$Y = \text{Executable}(G, P) \quad (3.110)$$

es apropiada cuando la instrucción contiene información suficiente, pertenece al dominio soportado y puede representarse mediante las capacidades disponibles.

Una respuesta:

$$Y = \text{Clarification}(q) \quad (3.111)$$

es apropiada cuando existe información faltante o una ambigüedad que puede resolverse mediante una respuesta breve del usuario.

Por ejemplo, una instrucción como:

“Bring it to her.”

puede requerir aclaración si los referentes de *it* y *her* no pueden determinarse a partir del contexto disponible.

Finalmente:

$$Y = \text{Rejected}(r) \quad (3.112)$$

es apropiada cuando la instrucción solicita una tarea que se encuentra fuera del dominio considerado, contradice las restricciones establecidas o requiere una capacidad no disponible.

Esta distinción permite establecer que:

$$\boxed{\text{producir un plan} \neq \text{responder correctamente en todos los casos}} \quad (3.113)$$

Forzar una secuencia de acciones ante una instrucción ambigua o no soportada puede constituir un error, aun cuando dicha secuencia sea estructuralmente válida.

3.5.5. Condición de control para la etapa simbólica

La separación entre interpretación y planificación de la arquitectura Qwen–CLIPS permite introducir una condición experimental destinada a aislar el comportamiento de la etapa simbólica.

En esta condición, la secuencia de objetivos de referencia se proporciona directamente a CLIPS:

$$\boxed{G_{\text{ref}} \xrightarrow{\text{CLIPS}} P_{\text{CLIPS}}.} \quad (3.114)$$

Esta condición no constituye una tercera arquitectura de planificación. Su propósito es evaluar la transformación:

$$G \rightarrow P \quad (3.115)$$

sin introducir previamente los errores que puedan originarse durante:

$$x \rightarrow G. \quad (3.116)$$

Si CLIPS produce un plan válido cuando recibe:

$$G_{\text{ref}}, \quad (3.117)$$

pero el flujo completo Qwen–CLIPS falla para la misma orden, el análisis puede dirigirse hacia la interpretación producida por Qwen.

En cambio, si:

$$V_P(G_{\text{ref}}, P_{\text{CLIPS}}, K, \mathcal{A}) = 0, \quad (3.118)$$

el problema corresponde a la cobertura o expansión de la etapa simbólica y no a la interpretación lingüística.

De esta manera, la condición de control proporciona un mecanismo para distinguir experimentalmente entre:

$$E_I : x \rightarrow G_{\text{incorrecto}} \quad (3.119)$$

y:

$$E_P : G_{\text{correcto}} \rightarrow P_{\text{incorrecto}}. \quad (3.120)$$

Esta separación constituye uno de los criterios utilizados posteriormente para clasificar los errores observados durante la evaluación.

3.5.6. Planificación y ejecución física

El último nivel considerado corresponde a la diferencia entre producir un plan conceptualmente válido y completar satisfactoriamente su ejecución sobre el robot.

Sea:

$$E \quad (3.121)$$

el resultado observado al intentar ejecutar un plan P .

Puede definirse conceptualmente:

$$V_E(P, E) \quad (3.122)$$

como el criterio que determina si la ejecución física de dicho plan alcanzó el resultado esperado.

Sin embargo:

$$\boxed{V_I = 1 \wedge V_P = 1 \not\Rightarrow V_E = 1.} \quad (3.123)$$

Un plan puede interpretar correctamente una instrucción y contener una secuencia coherente de acciones, pero su ejecución puede fallar debido a factores externos al proceso de planificación.

Entre ellos pueden encontrarse fallos de percepción, obstáculos durante la navegación, dificultades de manipulación, cambios en la posición de personas u objetos o problemas durante la interacción con el usuario.

Por ejemplo, un plan puede contener correctamente una acción destinada a tomar un objeto, pero la ejecución puede fallar si dicho objeto no es detectado o si el manipulador

no consigue sujetarlo.

De manera similar, una acción de navegación puede ser semántica y estructuralmente correcta, pero no completarse debido a una modificación del entorno físico.

Por tanto, el estado predictivo utilizado durante la planificación:

$$S_i^p \quad (3.124)$$

no debe confundirse con el estado efectivamente observado durante la ejecución.

Esta distinción permite interpretar los resultados experimentales sin atribuir automáticamente a la planificación errores que se originan en otros componentes del robot.

En consecuencia, la evaluación general del sistema se organiza conceptualmente en tres niveles principales:

$$\begin{array}{ccc} x & \xrightarrow{\text{interpretación}} & G \\ & & \downarrow \\ & & P \\ & & \downarrow \\ & & E \end{array} \quad (3.125)$$

correspondientes a la interpretación semántica, la generación del plan y la ejecución física.

Los criterios presentados en esta sección establecen únicamente la definición conceptual de validez utilizada en la investigación. Las métricas, umbrales, procedimientos de evaluación y condiciones experimentales concretas se desarrollan posteriormente en el capítulo de validación y experimentación.

Capítulo 4

Metodología y diseño de las arquitecturas evaluadas

4.1. Diseño metodológico de la investigación

4.1.1. Problema experimental

4.1.2. Variables y condiciones de comparación

4.1.3. Estrategia general de evaluación

4.2. Construcción del dominio experimental

4.2.1. Fuentes de conocimiento GPSR

4.2.2. Catálogos de personas, objetos y ubicaciones

4.2.3. Familias de tareas

4.2.4. Representación de los objetivos de referencia

4.3. Construcción del conjunto de datos

4.3.1. Generación de órdenes

4.3.2. Generación de objetivos canónicos

4.3.3. Variación lingüística

4.3.4. Balance y deduplicación

4.3.5. División de entrenamiento y evaluación

Capítulo 5

Implementación

5.1. Plataforma experimental

5.1.1. Robot Justina

5.1.2. Plataforma de cómputo

5.1.3. Entorno de software y ROS 2

5.2. Implementación del conjunto de datos

5.2.1. Generador de órdenes

5.2.2. Catálogos y CompetitionTemplate

5.2.3. Validadores

5.2.4. Dataset final

5.3. Entrenamiento e inferencia de Qwen2.5-7B

5.3.1. Preparación del modelo

5.3.2. Configuración QLoRA

5.3.3. Hiperparámetros de entrenamiento

5.3.4. Selección del checkpoint

5.3.5. Inferencia

5.3.6. Despliegue del modelo

Capítulo 6

Validación y experimentación

6.1. Diseño experimental

6.1.1. Conjunto de prueba

6.1.2. Procedimiento experimental

6.1.3. Repeticiones y control de variabilidad

6.2. Condiciones experimentales

6.2.1. Arquitectura Qwen–CLIPS

6.2.2. Arquitectura GPT directo

6.2.3. Condición de control con objetivos de referencia y CLIPS

6.3. Evaluación de la interpretación semántica

6.3.1. Exactitud de objetivos

6.3.2. Validez estructural

6.3.3. Exactitud de entidades y parámetros

6.3.4. Dependencias entre objetivos

6.4. Evaluación de la planificación

6.4.1. Acciones válidas

6.4.2. Consistencia del plan

Capítulo 7

Resultados

7.1. Resultados de la especialización de Qwen

7.1.1. Comportamiento durante entrenamiento

7.1.2. Evaluación del modelo seleccionado

7.2. Resultados de interpretación semántica

7.2.1. Qwen

7.2.2. GPT

7.2.3. Comparación

7.3. Resultados de planificación

7.3.1. Qwen–CLIPS

7.3.2. GPT directo

7.3.3. Condición de control CLIPS

7.3.4. Comparación

7.4. Análisis de errores

7.4.1. Errores de interpretación

7.4.2. Errores de planificación

7.4.3. Casos representativos

Capítulo 8

Discusión

8.1. Interpretación de los resultados respecto a la hipótesis

8.2. Interpretación semántica

8.2.1. Especialización de Qwen

8.2.2. Capacidad de GPT

8.2.3. Efecto de la variabilidad lingüística

8.3. Planificación de acciones

8.3.1. Expansión determinista mediante CLIPS

8.3.2. Planificación generativa mediante GPT

8.3.3. Separación entre errores de interpretación y planificación

8.4. Comparación Qwen–CLIPS frente a GPT directo

8.4.1. Exactitud

8.4.2. Consistencia

8.4.3. Latencia

Capítulo 9

Conclusiones y trabajo futuro

- 9.1. Conclusiones principales
- 9.2. Cumplimiento de los objetivos
- 9.3. Evaluación de la hipótesis
- 9.4. Contribuciones de la investigación
 - 9.4.1. Representación canónica de órdenes
 - 9.4.2. Arquitectura Qwen–CLIPS
 - 9.4.3. Comparación con planificación directa mediante GPT
 - 9.4.4. Metodología para separar errores de interpretación y planificación
- 9.5. Trabajo futuro

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Conclusiones Principales

10.2. Propuestas de Trabajo Futuro

- Fine-tuning de un LLM de código abierto para dominio específico
- Mejora de los mecanismos de seguridad y verificación
- Exploración de arquitecturas de integración más profundas

Referencias

- [1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.
- [3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.
- [4] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, “Chatgpt for robotics: Design principles and model abilities,” *Microsoft Research*, 2023.
- [7] NASA Johnson Space Center, “Clips reference manual,” tech. rep., NASA, 1991.
- [8] B. Pell *et al.*, “An autonomous spacecraft agent prototype,” in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.
- [9] J. Savage and otros, “Virbot: A modular architecture for service robots with hybrid planning,” in *RoboCup Symposium*, 2024.
- [10] L. Cheng, Y. Mao, and T. Ward, “Editorial: Neural network models in autonomous robotics,” *Frontiers in Neurorobotics*, 2025.
- [11] R. Raj and A. Kos, “An extensive study of convolutional neural networks: Applications in computer vision for improved robotics perceptions,” *Sensors*, 2025.
- [12] A. Sadek, *Building autonomous agents with hybrid navigation policies*. PhD thesis, Université de Technologie de Compiègne, 2024.

- [13] Alibaba Cloud Qwen Team, “Qwen official github repository,” 2026.
- [14] OpenAI, “Openai developer documentation and model guides,” 2026.
- [15] N. Houlsby, A. Giurciu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.
- [16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [17] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.
- [18] A. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.
- [19] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, “Neural-symbolic learning and reasoning: A survey and interpretation,” *arXiv preprint arXiv:1711.03902*, 2017.
- [20] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [21] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [22] D. Nau, T.-C. Au, O. Ilghami, U. Kuter, J. W. Murdock, D. Wu, and F. Yaman, “Shop2: An htn planning system,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 379–404, 2003.

- [23] K. Erol, J. Hendler, and D. S. Nau, “Htn planning: Complexity and expressivity,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 1123–1128, 1994.
- [24] I. Georgievski and M. Aiello, “An overview of hierarchical task network planning,” *Journal of Autonomous Agents and Multi-Agent Systems*, vol. 30, no. 4, pp. 709–752, 2016.
- [25] R. E. Fikes and N. J. Nilsson, “Strips: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [26] H. Geffner, “Model-free, model-based, and general intelligence,” *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 10–17, 2020.
- [27] R. Lallement, L. de Silva, and R. Alami, “Hatp: An htn planner for robotics,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 10–18, 2014.
- [28] L. de Silva, R. Lallement, and R. Alami, “The hatp hierarchical planner: Formalisation and an initial study of its usability and practicality,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5845–5852, 2016.
- [29] L. de Silva and R. Alami, “Dealing with on-line human-robot negotiations in hierarchical agent-based task planner,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 67–75, 2017.
- [30] R. Alami, R. Chatila, A. Clodic, *et al.*, “On human-aware task and motion planning,” *Robotics and Autonomous Systems*, vol. 61, no. 8, pp. 804–813, 2013.
- [31] X. Luo, S. Xu, R. Liu, and C. Liu, “Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6142–6149, 2024.
- [32] E. Gat, “On three-layer architectures,” in *Artificial Intelligence and Mobile Robots*, pp. 195–210, MIT Press, 1998.
- [33] F. Morales *et al.*, “Automated planning and scheduling in robot-aided rehabilitation: A review,” *Journal of NeuroEngineering and Rehabilitation*, vol. 22, no. 1, 2025.

- p. 45, 2025.
- [34] W. Pang, H. Li, Q. Huang, P. Li, and X. Ma, “Ontology-based mission planning for unmanned systems: A review,” *Journal of Unmanned Systems*, vol. 10, no. 2, pp. 112–128, 2022.
 - [35] Middlesex University, “Recognise act cycle,” 2023.
 - [36] B. L. Donnell, “Object/rule integration in clips,” *Expert Systems*, vol. 11, no. 1, pp. 29–45, 1994.
 - [37] F. J. Fanning, “An evaluation of an ada implementation of the rete algorithm for embedded flight processors,” *Air Force Institute of Technology Theses and Dissertations*, 1990.
 - [38] University of Žilina, “Production rules,” 2023.
 - [39] ScienceDirect, “Production systems and rules,” 2023.
 - [40] C. L. Forgy, “Rete: A fast algorithm for the many pattern/many object pattern match problem,” *Artificial Intelligence*, vol. 19, no. 1, pp. 17–37, 1982.
 - [41] C. L. Forgy, *On the efficient implementation of production systems*. PhD thesis, Carnegie-Mellon University, 1979.
 - [42] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee, “Robotic pick-and-place with natural language commands,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 4615–4622, 2020.
 - [43] L. Albert, “Average case complexity analysis of rete pattern-match algorithm, and average size of join in databases,” in *Foundations of Software Technology and Theoretical Computer Science*, vol. 405 of *Lecture Notes in Computer Science*, pp. 223–241, Springer, 1989.
 - [44] GitHub, “Attention is all you need - paper reading notes,” 2023.
 - [45] C. de desarrolladores de Alibaba Cloud, “Transformer modelos de lenguaje de nuestro tiempo: El desarrollo y la evolución tecnológica de los modelos de lenguaje a gran escala,” 2026.
 - [46] systems analysis.ru, “Transformer-architektur - der transformer,” 2025.
 - [47] A. Kashyap, “Recurrent memory-augmented transformers with chunked attention for long-context language modeling,” *arXiv preprint arXiv:2507.00453*, 2025.

- [48] D. Pollini, B. V. Guterres, R. S. Guerra, and R. B. Grando, “Reducing latency in llm-based natural language commands processing for robot navigation,” 2025.
- [49] C. Blog, “Qwen2.5-0.5b caso práctico: Guía para la construcción de un robot guía inteligente.,” 2026.